POLITECNICO
MILANO 1863

SCUOLA DI INGEGNERIA INDUSTRIALE
E DELL'INFORMAZIONE

Generative AI Models for Robotic Manipulation: a Comparative Evaluation in Virtual Environment

Tesi di Laurea Magistrale in
Mechanical Engineering - Ingegneria Meccanica
FA4 - Mechatronics and Robotics

Author: **Marco Porro**

Student ID: 249181
Advisor: Prof. Loris Roveda
Co-advisors: Angelo Moroncelli
Academic Year: 2024-25

Abstract

The rapid evolution of robotic manipulation is increasingly driven by generative Artificial Intelligence, transitioning from classical control to data-driven Imitation Learning via Foundation Models and Diffusion Policies. However, the dispersed nature of recent breakthroughs complicates fair performance comparisons. This thesis presents a rigorous comparative evaluation of four State-Of-The-Art architectures: Diffusion Policy, SmolVLA, π_0, and RDT-1B, to assess their capabilities, efficiency, and resource requirements within constrained computing environments. Utilizing the ManiSkill simulation framework, a unified demonstration dataset is created to train and evaluate the models across four fundamental table-top tasks: Push Cube, Pick Cube, Stack Cube, and Peg Insertion. The methodological approach prioritized reproducibility, evaluating not only success rates but also architectural trade-offs, data sensitivity, and generalization limits.

Results indicate that while Diffusion Policy excels in multi-modal action precision and resource efficiency, it lacks semantic reasoning and generalization capabilities. Conversely, Vision-Language-Action (VLA) models like π_0 and SmolVLA demonstrate generalization, instruction-following and contextual understanding but struggle with high-precision tasks like peg insertion. π_0 exhibited superior task comprehension and re-planning abilities compared to SmolVLA, at the cost of higher training time and hardware requirements. RDT-1B proved to be too resource-intensive for the constrained experimental setup, resulting in failed task executions. Ultimately, this work highlights that while Generative AI models offer unprecedented adaptability for robotic control, they still lack genuine problem-solving reasoning. By systematically analyzing these architectures, this thesis provides a valuable guide for deploying advanced robotic policies, creating a training dataset in simulation environment and reproducing the obtained results.

Keywords: Generative AI, Robotic Manipulation, Imitation Learning, Foundation Models, VLA, Diffusion Policy, ManiSkill Virtual Simulation Environment

Abstract in lingua italiana

La rapida evoluzione della manipolazione robotica è sempre più influenzata dall'Intelligenza Artificiale generativa, segnando il passaggio dal controllo classico all'Imitation Learning basato su grandi quantità dati tramite Foundation Models. Tuttavia, la natura frammentata delle recenti scoperte complica un confronto equo delle prestazioni. Questa tesi presenta una rigorosa valutazione comparativa di quattro architetture allo stato dell'arte: Diffusion Policy, SmolVLA, π_0 e RDT-1B, al fine di valutarne le capacità, l'efficienza e i requisiti in termini di risorse all'interno di ambienti computazionali con risorse limitate. Utilizzando il framework di simulazione ManiSkill, è stato creato un dataset unificato di dimostrazioni robotiche per addestrare e valutare i modelli su quattro compiti di manipolazione: Push Cube, Pick Cube, Stack Cube e Peg Insertion. L'approccio metodologico ha dato priorità alla riproducibilità, valutando non solo le percentuali di successo, ma anche i compromessi architetturali, la sensibilità ai dati e i limiti di generalizzazione.

I risultati indicano che, mentre la Diffusion Policy eccelle nella precisione delle azioni multi-modali e nell'efficienza delle risorse, scarseggia nel ragionamento semantico e nelle capacità di generalizzazione. Al contrario, i modelli VLA come π_0 e SmolVLA dimostrano capacità di generalizzazione, di comprensione delle istruzioni testuali e del contesto, ma faticano in compiti ad alta precisione come Peg Insertion. π_0 ha mostrato una comprensione del suo obiettivo e capacità di ripianificazione superiori rispetto a SmolVLA, a fronte però di maggiori tempi di addestramento e requisiti hardware. RDT-1B si è rivelato troppo oneroso in termini di risorse per il setup sperimentale vincolato, portando al fallimento nell'esecuzione dei task. In definitiva, questo lavoro evidenzia che, sebbene i modelli di IA generativa offrano un'adattabilità senza precedenti per il controllo robotico, mancano ancora di un vero e proprio ragionamento orientato al problem-solving. Analizzando sistematicamente queste architetture, questa tesi fornisce una guida dettagliata per l'implementazione di policy robotiche avanzate, per la creazione di un dataset di addestramento in ambiente di simulazione e per la riproducibilità dei risultati ottenuti.

Parole chiave: IA Generativa, Manipolazione Robotica, Imitation Learning, Foundation Models, VLA, Diffusion Policy, Ambiente Virtuale di Simulazione ManiSkill

Contents

Abstract **i**

Abstract in lingua italiana **iii**

Contents **v**

Introduction **1**

1 State of the art **5**

1.1 Foundation Models in Robotics 5

1.1.1 Transformer Architecture 6

1.2 Multi-Modal Foundation Models 7

1.3 Diffusion Models 7

1.3.1 Diffusion Policy 9

1.3.2 Flow Matching 9

1.4 VLAs 10

1.4.1 Pi0 11

1.4.2 SmolVLA 11

1.5 Generative Foundation Models 12

1.5.1 RDT-1B 12

1.6 Benchmarks 13

2 Methodology **15**

2.1 Simulation Environment 15

2.2 Dataset 16

2.3 Models implementation 19

2.4 Training & Fine-tuning 20

2.5 Evaluation modalities 21

3 Results **23**

3.1 Diffusion Policy 23
3.2 SmolVLA 25
3.3 Pi0 26
3.4 RDT-1B 28
3.5 Results comparison 29
3.5.1 Baselines 29
3.5.2 Trained checkpoints 30

4 Analysis and Ablation Studies 33
4.1 Task execution insights 33
4.1.1 Diffusion Policy 33
4.1.2 SmolVLA 34
4.1.3 Pi0 35
4.1.4 RDT-1B 36
4.1.5 General observations 36
4.2 Peg Insertion with variable clearance 37
4.3 Language instructions influence 38
4.4 Visual alteration influence 39

5 Conclusions, Limitations and Future Developments 43
5.1 Conclusions 43
5.2 Limitations 44
5.3 Future Developments 44
5.4 Available Material 45

Bibliography 47

A Benchmarks Leaderboard 55

B Hyperparameters 57

List of Figures 61

List of Tables 63

Introduction

The field of robotic manipulation is, with the rise of new technologies, currently going through huge changes; moving from classical control theory and specialized reinforcement learning techniques to a newer data driven imitation learning approach, focused on general purpose applicability.
The main protagonists of this transformation are foundation models, such as VLAs, which make use of large quantities of data to learn generalized representations of the world we live in, and Diffusion based policies, which can be considered complementary to them, though also leveraging Imitation Learning strategies.

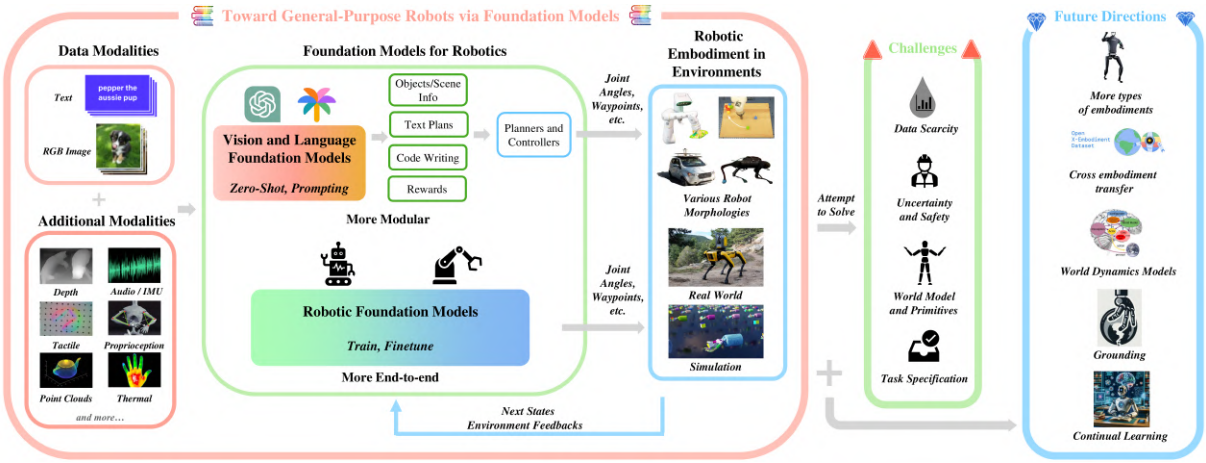


Figure 1: General-Purpose robotics workflow.

Vision-Language-Action models, such as π_0(pi0) [3] and smolVLA [37] represent the meeting point of three different fields: computer vision, natural language processing and robotic motor control. It is by fusing Large-Language-Models with physical embodiments that we allow robots to interpret and interact with their surroundings, reasoning on semantic concept before executing actions based on what they perceive.
The main advantages of these models lies in their potential to work zero-shot in never before seen environments and their ability to understand context.

At the same time, Diffusion Models together with their generative counterpart, Diffusion

Policy [9], have made progress on how robot handle continuous action spaces. Unlike traditional deterministic policies that output a single mean action, diffusion models learn the gradient of the data distribution, iteratively refining noise into precise trajectories. This specific property allows these policies to deal with multi-modal action distributions, characteristic of robotic problems, where multiple valid strategies are available to solve each single task.
Furthermore, architectures such as RDT-1B (Robotic Diffusion Transformer) [26] attempt to scale this generative capability by integrating diffusion action heads with transformer backbones capable of handling massive and heterogeneous datasets.

Problem Statement

Despite the theoretical advantages of these architectures, their practical applications and comparative evaluation still have a long way to go. The rapid pace of these innovations led to a disperse landscape where models are often evaluated on different datasets, different hardware and different environments, never taking into account ease of use and sub-optimal use cases, making it hard to compare.

Although large VLAs focus on generalization, they require prohibitive computational costs and result in high latency, making them difficult to deploy on standard robotic hardware. On the other hand, Diffusion Policies offer high precision and stability for very specific tasks, lacking semantic reasoning.
Moreover, the "black-box" nature of these deep learning models makes it difficult to evaluate how sensible they are to hyperparameter tuning and dataset quality.

Research Objectives & Contributions

The primary objective of this thesis is to conduct a rigorous, comparative evaluation of four State-Of-The-Art architectures: Diffusion Policy, π_0 (pi0), smolVLA, and RDT-1B.

To ensure a fair and scientifically valid comparison, this study utilizes the ManiSkill simulation framework [29], focusing on a mix of fundamental manipulation tasks including PickCube, StackCube, PushCube, and Peg Insertion. Rather than relying on pre-existing benchmarks, this research involves the construction of a single, unified small dataset for Imitation Learning and the development of a custom evaluation benchmark.

Beyond merely measuring success rates in the best case scenarios, this thesis contributes to the field of robotic learning by providing a systematic analysis of the operational characteristics of these models. We seek to answer not only which model performs best,

but at what cost. The investigation focuses on:

- **Architectural Efficiency**: Analyzing the trade-offs between model size and training/inference performance, eliminating variables that typically prevent a cross-paper fair comparisons.
- **Resource Requirements**: Assessing hardware requirements (specifically within constrained environments such as Google Colab) and time consumption for training and inference, identifying which models offer the best return on investment for resource-constrained research.
- **Data Sensitivity**: Understanding how different architectures respond to the quality and quantity of demonstration data, highlighting the practical challenges of data conversion and model adaptation.
- **Model Ease of Use**: Giving a personal judgment on the usability of the model, the documentation clarity and the code understandability from an external point of view.

Thesis Structure

The remainder of this thesis is organized as follows.
Chapter 1: reviews of the State of the Art literature regarding diffusion models, foundation models, imitation learning datasets creation and evaluation benchmarks.
Chapter 2: explains the experimental setup methodology, describing the generation of the unified ManiSkill dataset, the implementation and training of the models, and the comparative benchmark protocol used.
Chapter 3: presents the results of the experimental setup.
Chapter 4: interprets the empirical results critically, discussing .
Chapter 5: summarizes key findings and limitations, while proposing future development directions.

At the end of chapter 5, there will be an additional section providing all necessary material and resources to ensure complete reproducibility of this work.

1 State of the art

1.1. Foundation Models in Robotics

The field of robotic control is currently undergoing an interesting rapid shift in technologies, moving away from a "Sense-Plan-Act" methodology that used to be based on a highly specialized pipeline, composed of a perception, a planning and a control module; an approach that could be considered mathematically rigorous and logic but rigid at the same time, incapable of approaching unforeseen situations and fragile in front of the unpredictability and chaos of the real world.

It is only in recent years that Foundation Models have arrived as a solution to these issues. The term refers to the fact that these models are pre-trained on huge quantities of different data with the main scope of generalization and adaptability [4].
An approach that has been able to revolutionize computer vision at first, with the arrival of ImageNet [36], then computer language with Natural Language Processing (NLP), and now robotics, which can be considered as the meeting point of all these new discoveries and more.
Thanks to these single contributions, robotics is leaving behind explicit programming in favor of a data-driven learning style and the embodiment of Artificial Intelligence into physical agents, allowing robots to function also in new, chaotic environments.

Foundation Models make use of self-supervised learning [15] on large datasets to learn what we call transferable knowledge, meaning that what can be learned in one domain can then be used in another, similar to how humans work.
On the other part, there are many downsides, first of all the big size of these models, which leads to high computational costs, which leads to high hardware requirements and to slow inferences, making some models unusable in practice because of latency bottlenecks and thus low frequency control loops.
A second problem lies in the black box nature of these networks, which raises safety concerns and difficulties in the interpretations of errors known as hallucinations.
And finally, two indirect problems that cannot be ignored, which are the ecological prob-

lem due to the excessive usage of resources related to the power required by AI in general and the data-privacy problem correlated to the need for personal data, although this second one is less relevant in robotics.

The rise of Foundation Models also comes from the introduction of the Transformer architecture by Vaswani et al. [44], which made it possible to parallelize training over large datasets, replacing Recurrent Neural Networks (RNNs) [35] and LSTMs [17]. This architecture found its first use in Large Language Models (LLMs) like BERT [12] and GPT-3 [7] that showed that scaling data and model size led to emergent reasoning capabilities.

1.1.1. Transformer Architecture

The transformer is part of the deep learning architecture family with the peculiarity of processing data in parallel instead of sequentially. This function can be attributed to the self attention mechanism [44], which allows the tokens to talk to each others and the model to understand the importance of one element in relation to the others, instead of only itself.

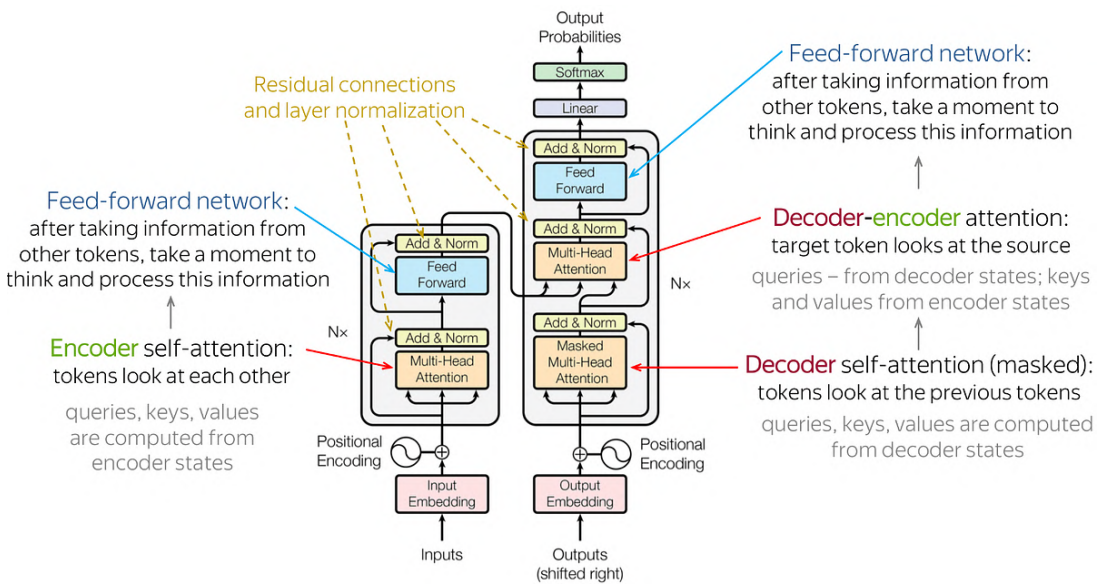


Figure 1.1: Transformer Architecture.

Its structure, depicted in Figure 1.1, is composed of different layers:

- **Tokenizer and Embeddings**: where inputs get divided into smaller units called tokens and then converted into embeddings, which are numerical values that are strictly correlated to the initial input.
- **Transformer Block**: divided into Encoder and Decoder blocks, although not both

are always needed, is the fundamental block that constitutes the self attention mechanism and the multilayer perceptron (MLP) which is a feed-forward network that, unlike attention, handles and refines each token representation singularly.

- **Output**: the final linear and softmax layer that converts the processed embeddings coming from the transformer block into probabilities values, essential to make predictions.

1.2. Multi-Modal Foundation Models

In the field of robotics, a middle stage between single-mode foundation models and the models implemented by this thesis can also be identified, and it is represented by Visual-Language-Models (VLMs) such as CLIP(Contrastive Language-Image Pre-Training) [32], which aligned visual and textual representations, essential for reasoning capabilities, or SayCan [1], which is a high-level planner that selected "skills"(pre-trained actions) from a list in order to complete a task based on semantic coherence and physical feasibility, without actually generating robotic actions.
Middle stage that was fundamental to arrive to Vision-Language-Action (VLA) models, like RT-1 [6] and RT-2 [5], now able to directly output low-level robotic control actions, making the robot move, by integrating the action policy into the Visual-Language Model.

A distinction needs to be made to introduce another branch of foundation models, parallel to the one of VLAs, generally identified as Generative Foundation Models, specifically based on diffusion processes.
Even though they can easily be compared and confused with VLAs, given their ability to also process visual and semantic input to then produce control actions as output, Generative Foundation Models like RDT-1B [26] and Octo [41] are structured differently, using the generative capabilities of denoising probabilistic models to prioritize continuous multi-modal representations over semantic reasoning.

1.3. Diffusion Models

To understand how low-level robotic control actions were achieved in the models reviewed in this work, a step back is necessary.
Whilst some foundation models, like OpenVLA [24], output discretized control actions that are quantized like tokens, others take advantage of diffusion models to output more fluid, continuous chunks of actions.

Diffusion Models, first introduced by Ho et al. with Denoising Diffusion Probabilistic

Models (DDPM) [16], learn data distribution through a process of stochastic denoising. First step of the process consists in gradually infusing our data sample x_0 with gaussian noise using a Markov chain.

$$q(x_t \mid x_{t-1}) = \mathcal{N}\Big(x_t;\ \sqrt{1-\beta_t}\, x_{t-1},\ \beta_t \mathbf{I}\Big)\,, \qquad t \to T \tag{1.1}$$

Second step consists of a neural network θ whose role is to try to learn how to reverse the first step by estimating the mean and variance of the gaussian distribution.

$$p_\theta(x_{t-1} \mid x_t) = \mathcal{N}(x_{t-1};\ \mu_\theta(x_t, t),\ \Sigma_\theta(x_t, t)) \tag{1.2}$$

What we obtain is therefore a neural network that is able to denoise a random distribution back to our original sample, which can be employed in a loss function that is simply the mean square error between the real noise ϵ and the predicted one ϵ_θ.

$$\mathcal{L}_{\text{simple}} = \mathbb{E}_{x_0,\epsilon,t}\left[\left\|\epsilon - \epsilon_\theta\big(\sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1-\bar{\alpha}_t}\, \epsilon,\ t\big)\right\|^2\right] \tag{1.3}$$

The first use that this algorithm found is in image generation, as we could train a neural network with a dataset of dog's pictures, for example, and it would learn how to generate a picture of a dog out of nothing by denoising pure random noise back to a distribution that resembled a dog, giving us an image that was similar to the one of the dataset but also different to all of them.

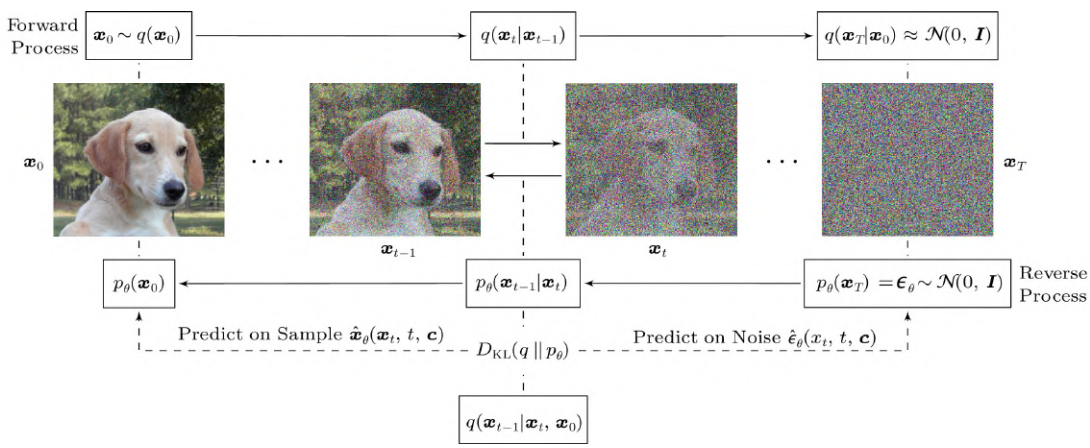


Figure 1.2: Graphical representation of the diffusion process [22].

1.3.1. Diffusion Policy

This tool only recently found an application in robotics thanks to Diffusion Policy [9][18]. Traditional methods of Imitation Learning such as Behavioral Cloning [42] were based on deterministic regression and couldn't deal with multi-modality; they would compute an average of the data, meaning that in robotics if there were two ways to do the same thing an average modality would be chosen, which often resulted in failure.

On the other hand, Diffusion Policy does not compute the mere average of the actions; by computing the gradient of the actions with the denoising diffusion process handled typically by a U-Net [34], multiple modalities of solving a task can be learned at the same time.
In addition, Diffusion Policy also achieved better temporal consistency through Temporal Action Chunking, that also used DDP and enabled it to predict entire trajectories instead of single actions at a time.

Compared to foundation models, diffusion policies lack semantic reasoning and multi-task abilities, but represent the best visual-motor policy in terms of action accuracy.
Furthermore, the lack of transformers makes it possible for diffusion policies to have a really small size, which allows their use even with fewer available resources, both in terms of time and hardware.

1.3.2. Flow Matching

An upgraded version of Diffusion Policy is represented by Flow Matching [25], used by pi0. As multiple denoising steps can lead to slow inference, Flow Matching uses a different mathematical formulation to solve the same problem in just one step.

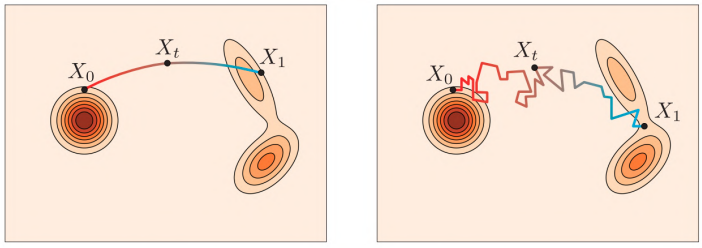
Instead of solving a stochastic differential equation (SDE), Flow Matching objective is to solve an ordinary differential equation (ODE) in which a neural network, instead of learning the denoising steps, learns the vector field $v_t(x)$ that directly connects the prior noisy distribution p_0 to the data distribution p_1 we want to train on, and then obtain in inference ($p_1 = actions$ in our case).

$$\frac{\mathrm{d}}{\mathrm{d}t}\,\phi_t(x) = v_t(\phi_t(x))\,, \qquad \phi_0(x) \sim p_0, \qquad \phi_1(x) \sim p_1 \tag{1.4}$$

Leading to a new Conditional Flow Matching loss in which u_t is the vector field we want to obtain.

$$\mathcal{L}_{\mathrm{CFM}}(\theta) = \mathbb{E}_{t,\, q(x_1),\, p(x_0)} \left[\| v_\theta(x_t, t) - u_t(x \mid x_1) \|^2 \right] \tag{1.5}$$

This results in a deterministic trajectory that should theoretically minimize the number of inference steps [27][14].



(a) Flow **(b)** Diffusion

Figure 1.3: Graphical interpretation of denoising steps comparison.

1.4. VLAs

As they are built upon VLMs, the standard VLA architecture is usually composed of the same blocks. A multi-modal tokenizer, that takes in input images and text instructions and uses specialized pre-trained encoders to process the data into tokens [13][38][30][10][43].
A Decoder-only Transformer that fuses and processes the input features by concatenating and refining the received tokens.
A final block, which takes the transformer output and converts it into robotic actions, an operation that can be performed in different ways.

Many of the first VLAs that innovated the previous state of the art, but also some modern important ones, employ the standard Next Token Prediction objective to generate discrete output actions, which often results in lower frequency control and more jittery trajectories [8][33][23].

A slightly different and newer branch of VLAs, like the one employed by this thesis, makes use of something called diffusion action heads.
Instead of tokenization, these action heads employ the diffusion or flow matching models to generate continuous vectors that lead to longer trajectories in opposition to single smaller actions. Other advantages of these diffusion action heads are increased long horizon stability and better temporal consistency.

1.4.1. Pi0

π_0, developed by Physical Intelligence, is a large-scale generalist VLA that innovated the field thanks to its ability to generate high frequency continuous actions (3.3 billion parameters and up to 50Hz), which was possible due to the adoption of a flow matching action head.
Pi0 also implements Paligemma [2], a large pre-trained VLM backbone, composed of the encoders SigLIP [50] and Gemma [40], and it is trained on: Vision-Language data from the web for semantic reasoning, Physical Intelligence Dataset and Open X-Embodiment [11], the largest open-source real robot dataset to date, allowing for cross-embodiment training.

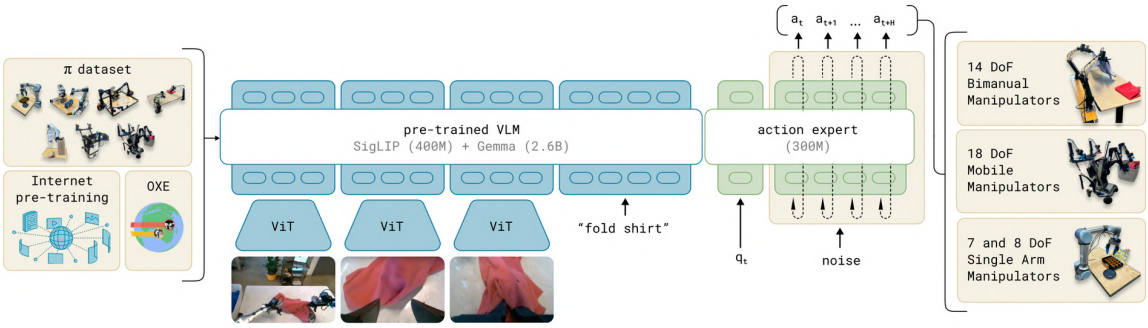


Figure 1.4: π_0 architecture.

Newer versions of π_0 are also available. They consists of Pi0-FAST [31], which makes use of Frequency-space Action Sequence Tokenization, a technique that transforms continuous token from time to frequency domain and increases precision in high-frequency tasks while speeding up training, and Pi05 [20], which tries to take generalization even further allowing robots to work in environments never seen during training.

1.4.2. SmolVLA

SmolVLA, developed by Hugging Face, followed the example of TinyVLA [46] and innovated VLAs by creating a more efficient compact model of just 450 million parameters without sacrificing significant performance.
It is designed to run on lighter, more affordable hardware, leveraging heavily low-rank adaptation (LoRA) [19], it uses a smaller version of SigLIP and Vicuna as backbone pre-trained encoders, and is trained using the open-source LeRobot dataset.
In addition, just like pi0, it employs a flow matching action head with temporal action chunking [47].

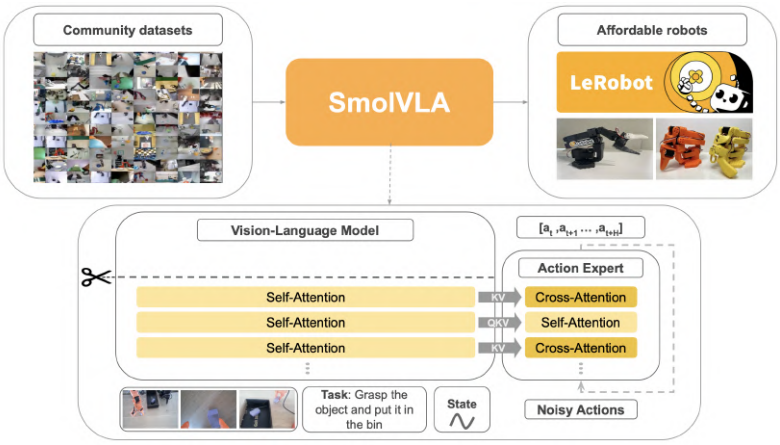


Figure 1.5: SmolVLA architecture.

1.5. Generative Foundation Models

1.5.1. RDT-1B

Finally, RDT-1B, as said before, its architecture differs both from the one of VLAs and the one of pure Diffusion Policies, as it is better described as a robotic diffusion transformer and represents the current largest diffusion-based foundation model with 1.2 billion parameters.

VLAs utilize the transformer to process and fuse linguistic and visual features, but then use a separate block to transform its output to actions; on the other hand, RDT-1B takes the diffusion principle and scales it using the transformer architecture, placing the diffusion at the center of the logic, while using the outputs of the visual and language encoders as a sort of conditioning.

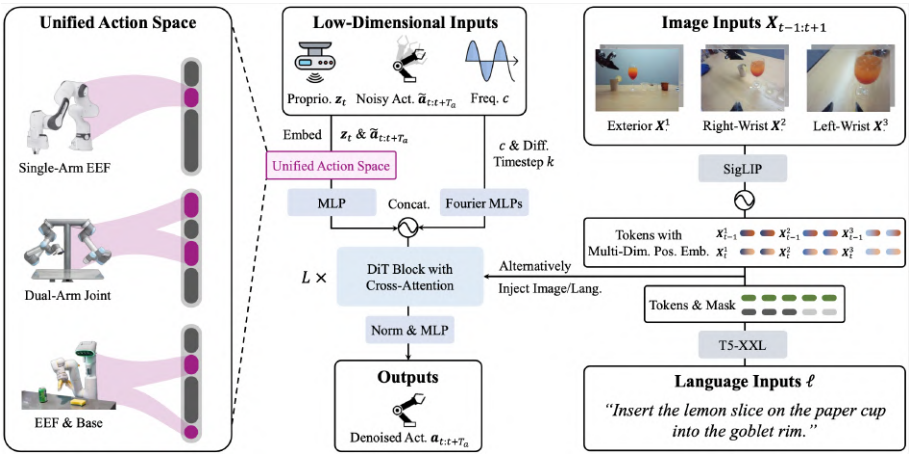


Figure 1.6: RDT-1B architecture.

RDT-1B also uses SigLIP as image encoder and T5-XXL [45] as text encoder. Furthermore, it is pre-trained on the largest multi-robot collection of 46 datasets, natively handling both bi-manual and wheeled robots, thus leading to broad zero-shot generalizability.
However, because of the employment of diffusion instead of flow matching, RDT-1B's inference latency is higher then the ones of π_0 and SmolVLA.

1.6. Benchmarks

A few different benchmark leaderboards can be found on the internet, showing for each model the average success rate on a variety of tasks mainly simulated in virtual environments.
Within each of these benchmarks, each reported model is theoretically trained or fine-tuned at the best of its capabilities and then evaluated on the same set of environments to ensure a fair comparison.

As different tests exist and there is not a single benchmark in which all models have been evaluated, a table is displayed below showing all the available data that regards the model taken into consideration in this thesis.
A more complete benchmark's leaderboard can be found in Table A.1 in Appendix A.

Table 1.1: Average success rates (%) across different benchmarks and papers.

Model Name	LIBERO	M-W	L+	RT2.0	MS*	RW**
π_0	**94.2**	47.9	53.6	**31.4**		**85**
SmolVLA	88.8	**57.3**				
π_0-FAST	85.5		**61.6**			
RDT-1B				24.1	**56.6**	
OpenVLA	76.5		15.6		4.8	15
Octo	75.1				0	5
TinyVLA		31.6				
Diffusion Policy	72.4	10.5		14.3	30.2	40

Note: M-W: Meta-World, L+: LIBERO+, RT2.0: RoboTwin 2.0, MS: ManiSkill, RW: Real World.
*ManiSkill benchmarks have all been conducted by the RDT-1B team.
**Real World comparisons have all been performed by the π_0 team.

From the table above, it can be observed that there is not a clear winner.

At first glance, it may seem like π_0 is the best performing model but it loses against SmolVLA and π_0-FAST in other benchmarks, showing discordant results, and it is only compared with RDT-1B once.
Various problems emerge, which are that these models seem to perform better in some environments rather than in others and that there are not enough direct comparisons to fairly compare them.
In addition, it is not possible to find details about how a lot of these results are achieved, which makes it impossible to reproduce the exact same results.

2 Methodology

Reviewing the literature, it has been shown that although comparative benchmarks do exist, their only purpose appears to be ranking the models by success rates without actually giving any additional information about observed behaviors, usability of the model and utilized parameters, missing out on the most practical aspects of their employment.

This section will describe the approach used by the thesis to create the dataset, implement the policies, train or fine-tune the models and then test them.

2.1. Simulation Environment

First thing, the ManiSkill 3 simulation framework has been selected for collecting the demonstrations that formed the dataset and as evaluation environment.
The reasons behind this choice rely on the fact that ManiSkill has been created explicitly as a benchmark platform for robot manipulation policies [39]; Even though it is still under development it already offers a wide set of pre-made manipulation tasks with the relative environments, motion-planning scripts and hugging face datasets.

Compared to other well known alternatives like MuJoCo, Isaac Gym, PyBullet or RLBench, ManiSkill offered the best mix of useful and practice features considering built-in benchmarks, generalization, reproducibility, realism, GPU scaling and, most importantly, compatibility and ease of use in a constrained environment such as Google Colab [48][28][21].
Every single code, training and evaluation has in fact been run on the browser-based cloud service that is Colab, which often makes complex programs like these hard or even impossible to use.
It is important to highlight the fact that, as this thesis wants to enforce the reproducibility aspect of the process, it made sense to select an open source platform and its ready-to-go tasks so that every single step can be easily replicated.

2.2. Dataset

The Dataset can be one the most important, if not the most important part of machine learning; after selecting ManiSkill as simulation platform a set of table-top two finger gripper tasks is chosen between all the possible ones, in particular this thesis selected the followings, with relative description and explanation:

- **Push Cube**: the objective is for the robot to close the gripper and push the blue cube by the side to a goal region in front of it. This task has been selected as an entry stage, it should represent the easiest task among the four, and it is a good indicator that the model is really learning and that the inference loop works as intended.

- **Pick Cube**: the objective is to grasp a red cube with the Panda robot and move it to a target goal position; however, in the official dataset the green target goal is not visible and, for this reason, we are considering the task successful if the robot picks and lifts the cube from the ground.
  Pick Cube is probably the most used task in machine learning manipulation, so it is perfect for a comparison and it also requires the model to understand better spatial coordination and temporal consistency.

- **Stack Cube**: the objective is to pick up a red cube and stack it on top of a green cube and let go of the cube without it falling. For success, it is also required that the red cube is on top of the green cube to within half of the cube size. Therefore, this task tests movement precision, sequential actions planning and the ability to recognize and distinguish similar but different objects.

- **Peg Insertion Side**: the objective is to pick up an orange-white peg and insert the orange end into the box with a hole in it. This is also a popular task that focuses on high accuracy as the hole is only slightly bigger then the peg.

A couple of modifications have been made to the scripts of these tasks to make them better compatible with the various models. The standard ManiSkill resolution of the cameras is set to 128×128, which is considered pretty low for the big models that are being used; specifically: SmolVLA mainly uses a resolution of 512×512, RDT-1B uses 384×384 and 224×224, while π_0 uses 224×224; because of this and because the image encoder that these three models use, SigLIP, is pretrained also on 224×224 images, this is the chosen resolution for the whole dataset. Diffusion Policy uses lower variable resolutions depending on the task difficulty so an equivalent 84×84 dataset is also tested.
The second modification regards the robot used, which is set to be the 7DOF Franka

Emika Panda robot for all tasks, whilst initially both StackCube and PegInsertionSide used also a wrist camera that made the tasks easier but also a lot heavier to train on.
All the other parameters are kept exactly the same, ensuring the tasks follow ManiSkill standards about camera position, observation space, initial position randomization and metrics calculations, both success criteria and reward function, which is not used during training but still gives us useful insights in the case the robot cannot execute the task to completion.

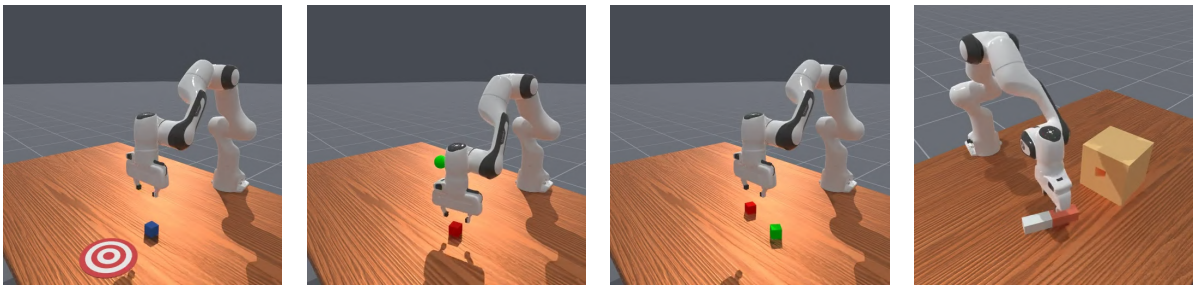
To recreate ManiSkill datasets two options are offered, either by using their pre-made motion-planning scripts to execute and save the demonstrations, or by using the `replay_trajectory` function, which transforms a fixed set of trajectories, available on their hugging face repository, in datasets with videos and statistics.
As the only modifications made to the environments regards the camera resolution it has been possible to use the latest method, modifying the function just slightly to change camera settings, thus obtaining a dataset that is easily replicable by anyone because it makes use of the exact same set of environments and initial conditions. Only rgb observations from the base camera are used.

Concerning the control modality, between the models used, RDT-1B is pretrained on `pd_joint_position` data, SmolVLA and π_0 are both pre-trained on a mixture of `pd_delta_ee_position` and `pd_delta_ee_position`, and Diffusion Policy just prefers position control over velocity, however all the models are able to deal with all the different types of control.
ManiSkill official trajectories were generated using `pd_joint_position`, and converting them to a different action space with the proprietary function can lead to failed executions, so the control mode has been left as is.

The number of required trajectories for the models to efficiently train is highly dependent on various factors such as task difficulty, scene variety, video frequency and dataset overall quality. Looking for guidelines to cases similar to this one, it is reported by the official respective papers that SmolVLA achieved good fine-tuning performances when using 50 different episodes, while Diffusion Policy used up to 200 episodes per task, so a compromise of 100 episodes per task is used, for a total of 400 episodes across the four tasks and 40 thousand individual frames.
Furthermore, the frequency used to capture the demonstration is $20Hz$ because that is the control frequency used by ManiSkill and because it is consistent with the models used. Using a frequency too high can lead to null actions that confuse the model and to repetitive frames that increase the computational cost without adding value; using a frequency too low can lead to loss of precision as small movements get lost.

As all the models apart from Diffusion Policy require also a text instruction in natural language as input, the following prompts were decided and added:

- **Push Cube**: "Push the blue cube towards the target".
- **Pick Cube**: "Pick up the red cube and bring it to the green goal".
- **Stack Cube**: "Pick up the red cube and stack it on top of the green cube".
- **Peg Insertion Side**: "Pick up the peg and insert it into the hole".

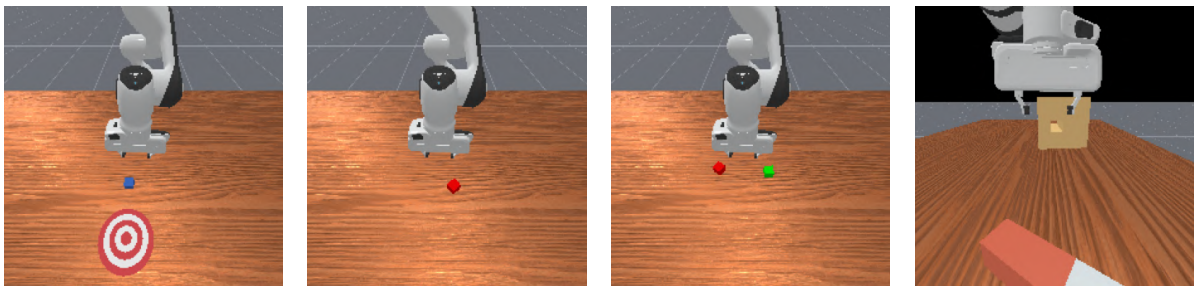


(a) Push Cube (b) Pick Cube (c) Stack Cube (d) Peg Insertion Side

Figure 2.1: ManiSkill tasks render

One particular detail that has been observed inspecting the dataset has to be noted, due to a bug in the "StackCube" environment script the very first frame of every episode sees the last joint, the one that connects the gripper, rotated by 90 degrees, which then quickly snaps into the correct position the frame right after. As the same script is also used to create the environment during inference, during evaluation the last joint starts in the wrong position too.

The base camera views that are used to train all the models are reported below with the original resolution of 224×224 to show exactly what the policies see.
By observing various episodes, it can also be observed that, when using top down views, the robotic arm can sometimes occlude the vision of the objects in the scene.
In the "Peg Insertion Side" task, as the initial position of the objects varies randomly, it can also happen for the peg to be not entirely visible inside the view, forcing the policy to grasp an object partially outside of its point of view. Furthermore, reaching the peg in certain position can also lead the arm to completely cover the camera.
The camera position is left as it is to leverage the official ManiSkill configuration and to observe the different models ability to adapt in these sub-optimal cases.



(a) Push Cube (b) Pick Cube (c) Stack Cube (d) Peg Insertion Side

Figure 2.2: ManiSkill tasks base camera view

The obtained model is then converted into "LeRobot Dataset v.3" format as it is easily stored in hugging face and as we already leverage the LeRobot framework to fine-tune some of the models.
To do that, ManiSkill already offers a conversion function, `convert_to_lerobot`, that is modified to implement the language instructions otherwise absent.

The LeRobot function `lerobot-edit-dataset` is then used to merge the datasets into a unified one and to add a necessary "version tag" (v3.0) that the ManiSkill version doesn't provide.
It must be also said that the official LeRobot Dataset Format documentation page seems to provide incoherent information. First one, the `meta/tasks.jsonl` file has been changed to `meta/tasks.parquet` in recent versions. Second one, the video folder structure is expected to be `videos/image_key/chunk-000/file-000.mp4` instead of what it is reported.

2.3. Models implementation

All the models have been implemented on Google Colab faithfully following the documentation, only slight modifications about the libraries versions are made to make them compatible with python 3.12 as the suggested python version 3.10 cannot be selected anymore on Colab.

In the case of Diffusion Policy both the original GitHub code from real Stanford and the LeRobot implementations are tested; however, the original code required more substantial changes to make it compatible with gymnasium and ManiSkill, although both works the final results are all obtained with the LeRobot implementation code.

In the case of RDT-1B, the memory offered by the A100 40GB GPU is not sufficient to be able to run the model and the language encoder, T5-XXL, at the same time. An

alternative official way is to pre-compute the language embedding separately, which is the chosen approach for this work.

2.4. Training & Fine-tuning

All the models are trained starting from the base pre-trained one, with the same hardware, which consists in the one offered by colab in the configuration plan with the A100 40GB GPU and 50GB of RAM and all the training and fine-tuning parameters are chosen following the ones suggested by the papers of the respective models and by the documentations.

Diffusion Policy is trained from scratch, as it is task specific, only on the "Stack Cube" task for resource constraints.
It is the only model trained with two slightly different datasets, in which the only differences are the camera resolution and the number of trajectories. The first dataset follows the same configuration as the other models, thus 100 demonstrations with an image resolution of 224×224 for comparison, while the second dataset follows the configuration of the official paper, thus 200 demonstrations with an image resolution of 84×84.

Some of the parameters are changed from the original configuration to make them coherent with the dataset size and properties, such as camera resolution, input state and number of actions to predict, while some others are changed to make them compatible with the available resources, like batch size, observation steps and action steps.

SmolVLA paper, explicitly state that for fine-tuning they not only freeze the vision encoder but also the entire backbone, training only the action expert, so the same procedure is used here.
On the other hand, π_0 doesn't say anything specific in that matter, for this reason the model is firstly trained freezing the backbone, until performances seem to have reached a plateau, then the weights are unfrozen and the vision encoder is fine-tuned as well, which is an approach aimed at reducing the risk of destroying the old knowledge of the encoder when fine-tuning on small datasets and to be able to document how much the model benefits from a full fine-tuning.

π_0 uses receding horizon control up to 50 observations and actions into the future, meaning that it is able to look at a chunk of frames, predict a sequence of 50 action steps and then use a sub-part of these actions before performing inference again.
However, this parameter is highly dependent on the control frequency, lower frequency translates into longer temporal prediction with the risk of losing temporal coherence; the

paper states that for a control frequency of 20Hz they use only the first 16 predicted steps, which is exactly what ManiSkill uses.
Even though it still makes sense to train on longer prediction horizon, pi0 has been trained directly on 16 action steps to greatly decrease the computational load, thus predicting 0.8 seconds into the future.

RDT-1B appears to be the most computationally intensive model between the four, the paper configuration for both pre-training and fine-tuning consists of a total of 48 H100 80GB GPU.

To make it work on the available setup the batch size is greatly reduced and gradient accumulation is used.
In addition, the model also looks at the last two time frames and predicts 64 actions into the future, of which 16 are executed. Contrary to π_0, here these parameters cannot be modified due to compatibility with the baseline architecture, so training cannot be accelerated.
Lastly, LoRA is not yet implemented and full fine-tuning is the only available option.

2.5. Evaluation modalities

The first test that is conducted is an offline action accuracy test, to verify that the policy actually learned the connection between observations, instruction and actions; five hundred different frames are randomly extracted, using the seed 420, from the training dataset and fed into the model to generate the action that will be compared. Specifically, mean absolute error, mean squared error and cosine similarity are computed.

The main evaluation consists of a rollout of the model inside the ManiSkill environment with randomized but reproducible initial conditions, so that each model is fairly evaluated on the exact same set of environments. From here, success rates and average rewards are computed.
The inference code written to conduct the test imitate the evaluation function of each individual framework to ensure that the policy is deployed as intended.

Model baselines are also tested inside the same benchmark to be able to compare the performances before and after the fine-tuning.
Lastly, a number of tests are also conducted to analyze how the different models behave in unintended scenarios; these tests include trying different or incoherent text instructions, and feeding the policy altered or occluded scene views.

3 Results

After each model has been trained, or fine-tuned, on the ManiSkill dataset, the best checkpoint is chosen. To select it, two tests are conducted, firstly the offline action accuracy test is performed to make sure the policy is getting increasingly better the higher the steps; secondly, a few tasks rollout are performed on never seen environments to make sure the policy hasn't over-fitted to the new data.
As expected, the best checkpoint is identified in the last one for all trainings since the number of training steps used has been always lower then the suggested one because of time constraints.

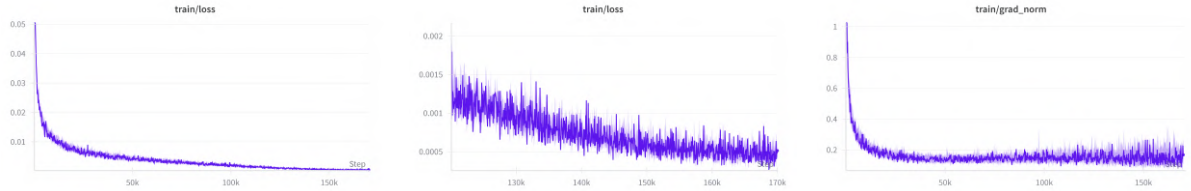
The final evaluation test inside ManiSkill consists of 50 trials for each task on a single reproducible seed, which is 123, in which the best policy checkpoint for each model is deployed, success rates are automatically computed and videos are inspected for additional insights. A maximum of 200 steps is allowed for every task trial.

3.1. Diffusion Policy

Diffusion Policy is trained from scratch on the "Stack Cube" task with two slightly different configurations.

In the first configuration, the policy is trained on a dataset that shares the exact same characteristics with the dataset used to train all the other models in this thesis, for comparison purposes.
Therefore, 100 episodes of stack cube with image resolution 224×224, randomly cropped to 200×200 are used. The policy is trained for 1000 epochs, that corresponds to 170 000 steps with a batch size of 64 for almost 7 hours, all the other training parameters are kept consistent with its paper configuration and are reported in Appendix B, Table B.1.

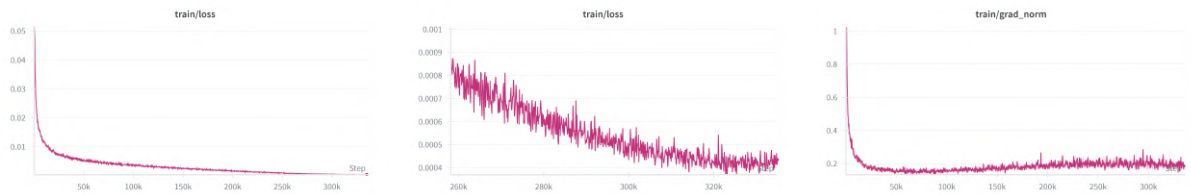
The training metrics are reported in Figure 3.1 with a zoom on the last portion of the loss trend for better visibility.



(a) Loss (b) Loss zoom (c) Gradient norm

Figure 3.1: Diffusion Policy training metrics.

In the second configuration, all the parameters suggested by the Diffusion Policy paper are used, with the only exception of the epochs number which they set to 3 000 epoch. Thus, 200 episodes of stack cube are used, with an image resolution of 84×84 cropped to 76×76, while the policy is trained for 1 000 epochs, for a total of 335 000 steps with batch size 64 and 9 hours on A100 40GB GPU. Metrics reported in Figure 3.2.



(a) Loss (b) Loss zoom (c) Gradient norm

Figure 3.2: Diffusion Policy training metrics with paper dataset configuration.

Looking at the training metrics, it's observed that both trainings follow a very similar trend.
For the first configuration, the loss is converging to the value 0.0005 within the last 20k steps and the gradient norm is stable around 0.1 but getting noisier after epoch 500.
For the second configuration, the loss is converging to the value 0.0004 within the last 20k steps while the gradient norm is less noisy but starts increasing after 300 epoch and then converge to 0.2 value.

The behaviors of the gradient norm are not too unusual for a diffusion model, it could mean that the model has learned the general low frequency trajectory and is then trying to learn the high frequency control and the noise, which is coherent with the findings of the Diffusion Policy paper.
As it could also indicate overfitting to the data distribution, every intermediate checkpoint is tested with different initial conditions, which shows that the best checkpoint is still the last one and thus there is no overfitting.

In Table 3.1 are reported the results of the offline action accuracy test to compare the results of both configurations. The metrics reported include the Mean Absolute Error (MAE), the Mean Squared Error (MSE) and the Cosine Similarity (COS), while values are presented as Mean $\pm$ Std.

Table 3.1: Action Accuracy Test of Diffusion Policy

Metric	**StackCube** (224×224)	**StackCube** (84×84)
MAE ($\downarrow$)	0.012 ± 0.034	0.024 ± 0.065
MSE ($\downarrow$)	0.004 ± 0.033	0.022 ± 0.101
Cosine Similarity ($\uparrow$)	0.999 ± 0.009	0.994 ± 0.028

The results of the test seem to indicate that the first configuration led to more accurate action predictions, even though this is incoherent with the success rates obtained evaluating the models inside ManiSkill and reported below in Table 3.2
This is the only case in which the offline and the online tests show discordant results.

Table 3.2: Evaluation Success Rates of Diffusion Policy

Metric	**StackCube** (224×224)	**StackCube** (84×84)
Success Rate	18%	30%

Following the official configuration, during inference Diffusion Policy looks at the last two timesteps, predicts 8 action steps and then executes 4 in a row. Additional insight about the task execution are discussed in Chapter 4.

3.2. SmolVLA

SmolVLA is fine-tuned starting form the base model that they suggest of 450 millions parameters for 40 000 steps with a batch size of 64, taking almost 10 hours.

Here, the image tokenization resolution is padded to 512×512 to align with the pre-training, while all the other parameters are kept as suggested by the paper. So we adopt an action horizon of 50 steps and train only the action expert and the state projection, keeping everything else frozen.

The other useful training parameters are listed in Appendix B, Table B.2, while the metrics from $W\&B$ are reported in Figure 3.3.

(a) Loss (b) Loss zoom (c) Gradient norm

Figure 3.3: SmolVLA training metrics.

The loss converged to value 0.004 and the gradient norm decreased monotonically, flattening only in the last couple thousand steps.
Action accuracy test results are shown in Table 3.3.

Table 3.3: Action Accuracy Test of SmolVLA

Metric	**PickCube**	**StackCube**	**PushCube**	**PegInsertion**
MAE (↓)	0.133 ± 0.186	0.044 ± 0.069	0.052 ± 0.071	0.040 ± 0.100
MSE (↓)	0.175 ± 0.258	0.020 ± 0.074	0.020 ± 0.036	0.044 ± 0.147
Cosine Similarity (↑)	0.938 ± 0.092	0.994 ± 0.023	0.994 ± 0.011	0.985 ± 0.051

Reported in Table 3.4 are the success rates for the ManiSkill evaluations.

Table 3.4: Evaluation Success Rates of SmolVLA

Task	**Success Rate** (%)
PushCube	100
PickCube	74
StackCube	8
PegInsertion	2

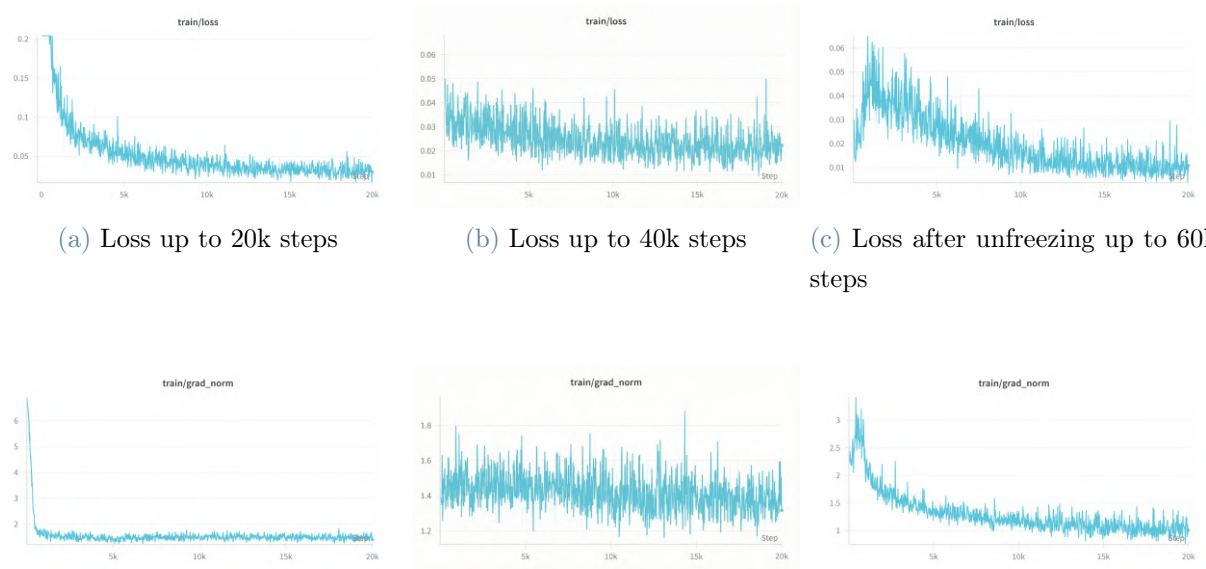
Even though SmolVLA can predict up to 50 action steps, the paper methodology says to make it re-plan the entire trajectory after every single step in simulation, which is what is done here too.

3.3. Pi0

π_0 is fine-tuned starting from the base model for a total of 60 000 steps with batch size 32, of which the first 40 thousand are done keeping the VLM backbone frozen and training only the action expert, and the last 20 thousand consist of full fine-tuning, as PEFT is

not used.
The official documentation suggests setting the compiler to $True$ to speed up the training of about 20%, but it proved to be unstable on Colab, leading to random crashes, thus it was disabled and the complete training lasted 22 hours.

As explained in the Methodology chapter, the action horizon has been brought from 50 to 16 steps because of resource constraints, the other parameter used are in Appendix B, Table B.3. Reported in Figure 3.4 are the metrics of loss and gradient norm each for 20 thousand steps, since the training had to be divided into three parts because of Colab's session time constraint.



(a) Loss up to 20k steps (b) Loss up to 40k steps (c) Loss after unfreezing up to 60k steps

(d) Gradient norm up to 20k steps (e) Gradient norm up to 40k steps (f) Gradient norm after unfreezing up to 60k steps

Figure 3.4: Pi0 training metrics.

At 40k steps, with the VLM weights still frozen, the loss function converged to the value of 0.02 and the gradient norm remained stable at 1.4; after unfreezing the weights and restarting the training, there is a spike in both metrics because the gradient needed to be readjusted, after that the loss reached a lower value of 0.01 and the gradient norm that of 1, leading to much better action accuracy scores, listed in Table 3.5.

Success rates for the ManiSkill tasks reported in Table 3.6.
Also for π_0, we predict a sequence of 16 actions, of which 15 are discarded and only the first one is executed.

Table 3.5: Action Accuracy Test of π_0

Metric	PickCube	StackCube	PushCube	PegInsertion
MAE (↓)	0.048 ± 0.117	0.017 ± 0.045	0.031 ± 0.085	0.018 ± 0.064
MSE (↓)	0.055 ± 0.161	0.007 ± 0.050	0.022 ± 0.119	0.016 ± 0.101
Cosine Similarity (↑)	0.980 ± 0.059	0.998 ± 0.015	0.993 ± 0.040	0.994 ± 0.034

Table 3.6: Evaluation Success Rates of π_0

Task	Success Rate (%)
PushCube	98
PickCube	86
StackCube	10
PegInsertion	2

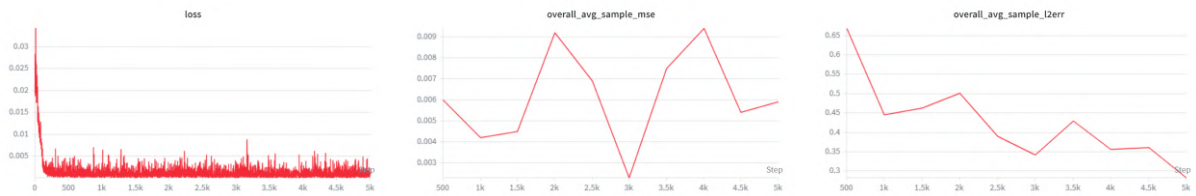
3.4. RDT-1B

RDT-1B is fine-tuned, starting from their pre-trained checkpoint, for only 5000 steps with batch size 8 and gradient accumulation factor of 4, to mimic a batch size of 32 which is otherwise unobtainable with the available setup.

Apart from image resolution and number of cameras, all the other parameters are kept coherent with the paper suggestions and reported in Table B.4, in Appendix B.

Although the batch size is greatly lowered and language embeddings are pre-computed, the GPU memory is almost saturated and random unexpected crashes happened during fine-tuning for unknown reasons.
In addition to this, resuming the training from the obtained checkpoint resulted extremely impractical due to very high data leakage which led the system RAM to get saturated and the Colab session to crash after a few thousand steps; this last issue made it impossible to complete the trainings on Google servers and led the model to be under-trained.

Because of this, the results of RDT-1B are not very representative of the model true capabilities and the comparison is not to be considered fair.



(a) Loss (b) MSE (c) L_2 loss

Figure 3.5: RDT-1B training metrics.

From the metrics reported in Figure 3.5 it is observed that the loss rapidly drops and then stabilizes around value 0.001 with a lot of fluctuations, typical of this type of diffusion architecture. The Mean Square Error metrics and the L_2 loss are also automatically computed and show, respectively, a fluctuating and a decreasing behavior.

It is safe to assume that the model would have benefited from a much higher number of training steps and it can be especially observed in evaluation as the policy does not achieve any success due to a particularly low accuracy.

3.5. Results comparison

3.5.1. Baselines

The baselines of these models are also tested inside the same evaluation benchmark, if possible, to compare also the difference in performance before and after fine-tuning.

Not surprisingly, the baselines achieved a consistent success rate of 0% in all tasks. Result that is easily explained by the fact that the ManiSkill demonstration data is too far from the pre-training distribution of these models and generalization capabilities are not advanced enough.

SmolVLA and π_0 in particular, are pre-trained for the most part on real world data, resulting in a wide sim-to-real gap. Additionally, even if it is possible that the simulation environment data is present in the pre-training dataset, different camera settings can confuse the model, as it is shown later in Chapter 4, and it is not known whether any of these models is used to top-down views.

3.5.2. Trained checkpoints

For a more direct comparison, the success rates of all the models tested are reported in Table 3.7, while additional relevant information are reported in Table 3.8

Table 3.7: Comparative evaluation success rates (%) across different models.

Model	PushCube	PickCube	StackCube	PegInsertion
RDT-1B	0	0	0	0
SmolVLA	100	74	8	2
π_0	98	86	10	2
Diffusion Policy (224p)	-	-	18	-
Diffusion Policy (84p)	-	-	30	-

Table 3.8: Training specifications and resource requirements.

Model	Train Time	VRAM	Train Samples	Model Weight
RDT-1B	3h 20m	38.8 GB	0.16×10^6	20 GB
SmolVLA	9h 20m	21.6 GB	2.5×10^6	1.3 GB
π_0	22h	33.6 GB	1.9×10^6	18 GB
Diffusion Policy (224p)	6h 45m	9.6 GB	1.1×10^7	3 GB
Diffusion Policy (84p)	8h	6.4 GB	2.1×10^7	3 GB

"Train Samples" corresponds to the total number of frames seen by the policy during training.

Looking at the success rates, it is clear that Diffusion Policy is the most capable model in terms of task completion and accuracy, especially when leveraging the dataset configuration suggested by the paper.
Followed then by π_0 and SmolVLA, which achieves slightly inferior performance, and lastly RDT-1B, which failed to achieve any success, supposedly because of an insufficient training, limited by the available resources.
It's important to underline the fact that Diffusion Policy remains a single-task policy, which gets trained from scratch on a specific task, and therefore achieves higher precision sacrificing all the generalization and few-shot learning abilities of foundation models.

Because of this, Diffusion Policy is also the most lightweight model, requiring far less time and hardware resources than all the other models to achieve the same success rates, even though its inference times are higher than those of SmolVLA and π_0.
SmolVLA proves to be an incredibly efficient model, requiring less RAM, training and

inference time, while achieving good performances.
π_0 is the best performing foundation model with affordable resources, while RDT-1B resource requirements are too elevated and its inference times extremely slow.

The LeRobot framework, used by SmolVLA, π_0 and Diffusion Policy, together with its documentation, offer the most user-friendly approach in training these policies and benefits from continuous GitHub support.
The original Diffusion Policy code is modular but also intricate and requires more experience and understanding. Depending on the simulator, the code may also require several modifications, as it is outdated and not natively compatible with gymnasium.
RDT-1B code is pretty straight forward but also less clear in its documentation and presents a bunch of features that are still not implemented, such as gradient check-pointing and LoRA.

4 | Analysis and Ablation Studies

4.1. Task execution insights

In this chapter, the observed behaviors of the models are discussed, together with some additional experiments to test the policies limits, and then an use case comparison is given.

For all models, the bug related to the gripper initial position in the "Stack Cube" environment script didn't seem to lead to any problem. At the start of the task execution, every policy would simply rotate the last joint of 90 degrees smoothly, reaching the position seen in the demonstration.

4.1.1. Diffusion Policy

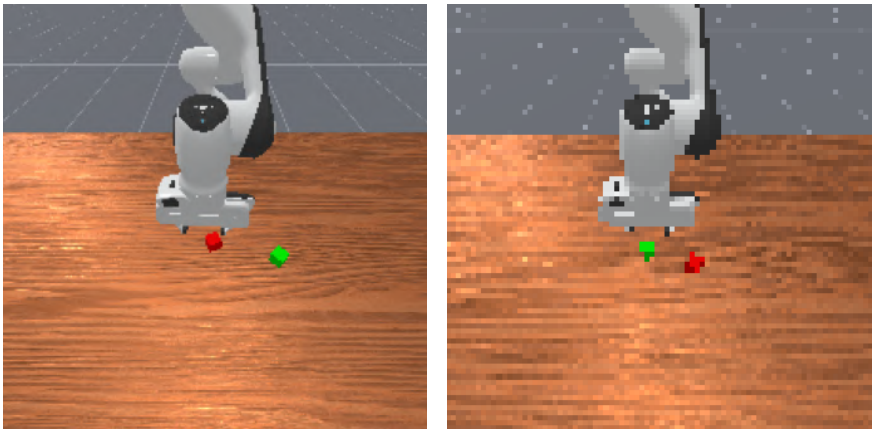
Both diffusion policies configurations are characterized by very smooth movements of the robotic arm, representing the most natural looking actions.

A common problem in the execution of "Stack Cube" is the difficulty in orientation whenever the two cubes were really close to each others, often trying to grasp somewhere in between them and other times colliding with the green block.

Another common problem is getting stack in a loop after failing to initially grasp the block, leading to the gripper continuously trying to grasp in the wrong position or to the arm going back and forth between the two cubes.

Regarding only the first configuration (100 episodes with image resolution 224), the main struggle is in correctly picking up the red cube; once the red cube has been successfully lifted, it will usually also be placed on top of the green one correctly.
In addition, a particular behavior has been observed, as in 14% of the trials, after failing to grasp the red cube, the arm would then reach the green cube, pick it up and then try to stack it on top of the red one. This behavior is still present in all the models, but not nearly as much.

When testing the second configuration of diffusion policy (200 episodes with image resolution 84), the best re-planning capabilities has been observed, with a highly reactive policy and successfully completing an action after having failed the first time.
Here, the main problem is not to pick up the red cube anymore, instead it's to correctly place it on top of the other. In many trials the robot correctly positions itself above the green cube but then quickly moves away before dropping the cube, in other trials the cube is dropped too early or dropped and then re-picked so fast that the task doesn't end.



(a) StackCube 224×224 (b) StackCube 84×84

Figure 4.1: Diffusion Policy image resolution comparison.

Comparing the performances and results of these two configurations, which have comparable training times and equal number of training epochs, the findings are aligned with those of the original paper of Diffusion Policy.
Even though the images in 84×84 pixels are almost hard to interpret, the policy benefits much more from a higher number of demonstrations compared to a higher resolution.

4.1.2. SmolVLA

SmolVLA also achieved pretty smooth trajectories.
"Push Cube" is generally completed correctly, although it happened a couple of times to slide the cube touching it from the top instead of from the side like in the demonstrations.

During "Pick Cube", the robot shows no sign of re-planning after failing to grasp the cube, which always happens either because the gripper orientation is wrong or because it is not correctly centered above the cube.

In "Stack Cube", the same difficulties of diffusion policy emerge. The policy is often confused by the presence of the two cubes, sometimes grasping the area in between them,

other times trying to pick up the green cube after having failed with the red one.
One notable difference from "Pick Cube" is that here the model tries to re-plan after failing; however, the re-planned action is almost always unsuccessful and it is not as reactive as diffusion policy.

With "Peg Insertion" several problems arise.
When the peg coordinates are too close to the camera view, and the peg is partially cut from the camera, it becomes very hard for the robot to grasp it. Only in a few cases, it correctly grasped the peg outside of the camera view.
Thicker pegs are harder to grasp because of wrong positioning, while longer pegs are harder to handle, sometimes the gripper is not closed enough and the peg rotates, other times the end of the peg collides with the block in front with the side, resulting in the policy getting stuck as it does not know how to handle that problem.
Lastly, even though there were trials in which the peg was extremely close to the hole but ultimately failed, there are also a lot of trials in which the peg positioning seems completely random.

4.1.3. Pi0

In the only trial of "Push Cube" that π_0 failed, the gripper hit the top of the cube, pushing it aside, and then proceeded with the task execution without the cube.

Also π_0 showed no capabilities of re-planning in the task "Pick Cube" after failing to grasp the cube.
In multiple attempts, after closing the gripper without being centered with the cube, the robot would then forget the instructions and start executing movements associated with "Push Cube" even though there was no target area.

"Stack Cube" is characterized by more uncertain movements from the robot, but re-planning is observed a lot.
The policy sometimes suffers from the same common problems; when cubes are too close to each other it gets confused, after failing to grasp the red cube the robot tries to lift the green one, the red cube is dropped aside the green one and not on top of it.

However, even if the success rate is just slightly higher then the one of SmolVLA, π_0 is much closer to successful executions then the other model.
Although SmolVLA almost never succeeded in grasping the first cube, Pi0 often fails later on when placing the cube, showing overall better task understanding.

Very similarly, during "Peg Insertion" the same behaviors are observed, but π_0 doesn't

struggle as much as SmolVLA. In an higher number of trials the peg in very close to the center of the hole and it doesn't look like the policy is trying to guess the location of the hole.

4.1.4. RDT-1B

It can be stated that the training pipeline is working correctly as huge improvements can already be seen when comparing the policy with its baseline; the actions performed seem coherent with the environment although very imprecise, and robotic movements are incredibly fluid and fast compared to the other models, likely because of the long action horizon.

On the other hand, the diffusion process associated with the transformer architecture makes also the inference process much slower than the other policies, with a complete evaluation loop taking up to 3 hours and 20 minutes for 200 trials of 200 steps each, more than three times the duration of π_0.

To inspect the influence of the action horizon length on the robotic movements, each task is performed once with a decreasing number of executed actions while predicting the fixed amount of 64.
By dividing the action chunks from 16 to 8, and from 8 to 4, the accuracy increases and actions are less prone to overshoot; at the same time, movements become less smooth, the arm tend to tremble a lot and inference times doubles.
If the number of executed steps is brought down to 2 or 1, the movements become extremely jittery and the robot easily gets stuck in loops.

4.1.5. General observations

Although limited, "Push Cube" showed some signs of possible overfitting with π_0 as it executed the actions blindly once, and even inside the wrong environment with the wrong natural language instructions. Which may be given by the fact that the task is easier compared to the others and often very repetitive.

A common problem in the execution of "Stack Cube" is the difficulty in orientation whenever the two cubes were really close to each others, having episodes in which, to grasp the red cube, the gripper would collide with the green one. Highlighting the fact that the models are learning the correlation between the gripper position and the block to grasp coordinates, without actually reasoning about the rest of the environment and possible collisions.

A counterintuitive phenomenon emerges when using top-down camera setups. Despite the difficulty of estimating depth from this angle, the models are extremely accurate at vertical positioning. In contrast, they struggle significantly in aligning with objects correctly on the table.
A possible explanation to this phenomenon is that the robot arm is always placed at the same height when hovering and when grasping, so it learned very well at what depth to go using proprioceptive data, instead of relying on visual data.

Another big problem that can be observed during the tests is that, even though VLAs possess generalization capabilities, they do not really possess problem solving or general reasoning capabilities.
This behavior becomes obvious whenever we see the robot trying to execute the same actions in loop without ever trying to change its strategy, or when the peg collides with the block, ending up in an unforeseen situation that the policy does not know how to handle.

4.2. Peg Insertion with variable clearance

"Peg Insertion" is the harder task between the tested ones in terms of required accuracy, and the fact that both π_0 and SmolVLA succeeded in at least one trial testifies the potential of these models.
To better differentiate the abilities of the two models, however, the hole size is increased and the task is tested again following the same procedure of 50 attempts with variable initial conditions.

The peg thickness can vary between 3 and 5 centimeters while the standard clearance is always of 3 millimeters, which means that the hole size is always only 6 millimeters wider than the peg. Simulations are performed again with an increasingly bigger clearance to see how the models adapt, related results are reported in Table 4.1.

Table 4.1: Peg Insertion success rates with variable clearance.

Model	**3 mm**	**6 mm**	**10 mm**
π_0	2%	8%	8%
SmolVLA	2%	2%	4%

π_0 performance initially increases when doubling the clearance, but then does not increase anymore when switching to $10mm$. While on the other part, SmolVLA performance increases just slightly only when the hole becomes much bigger.

This results are not obvious because there are a lot of trials in which the peg is extremely close to the center of the hole when it is smaller, so it is expected for the peg to be inserted correctly easily ones the hole is increased in size. Instead, with bigger holes the pegs still get stuck just by a tiny fraction with the difference that now they are even more off-centered.

The only plausible explanation to this phenomenon is that the policies are not really aiming for the center of the hole, instead they are just taking as a reference the outlines of the hole and shifting of the expected clearance from there.

4.3. Language instructions influence

To also test generalization capabilities of the LLM backbone and how much different natural language instructions impact task execution, various alternatives are used and listed below. In bold, the original instructions for reference.

Push Cube: "Push the blue cube towards the target"

- "Slide the blue object into the goal area"
- "Can you please push that blue block over to the target?"

Pick Cube: "Pick up the red cube and bring it to the green goal"

- "Grasp the block"
- "Lift the block"

Stack Cube: "Pick up the red cube and stack it on top of the green cube"

- "Lift the red cube, then place it onto the green one"
- "Stack the cubes"

Peg Insertion Side: "Pick up the peg and insert it into the hole"

- "Plug the peg in the hole"
- "Insert the plug in the center of the hole"

Since the findings are very similar for both π_0 and SmolVLA the comments below are valid for both models.

In general, using a natural language instruction different from the one used in the training dataset resulted in worse performance for all tasks, in particular for the task "Push Cube". Robotic movements become more uncertain and slower, and also accuracy diminishes. In

some rare cases, it looks like the tasks are being mixed up. For example, when using the word "goal" inside the "Push Cube" task, the robot tends to try to grasp the cube and executes small vertical movements, supposedly because that word is present in the training instruction of "Pick Cube".

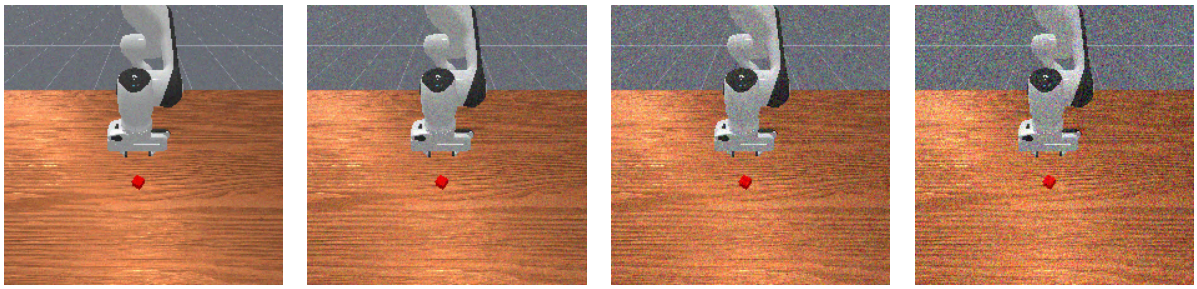
As an additional experiment, by inverting the tasks instructions a different observation can be made.
If the "Push Cube" prompt is used inside the "Peg Insertion Side" environment, the policies will ignore it and still perform the task only based on what they see.
If instead, the "Push Cube" prompt is used inside a similar looking environment like "Pick Cube", the robot will push the cube to a non-existing target area.

This means that the models are not willing to execute an instruction that is not coherent with what they expect from the training data, unless the context is similar enough to confuse them.

4.4. Visual alteration influence

To evaluate how well the models can adapt to visual disturbances, occlusions of the scenes and slight view variations, three more tests are performed. Here, the evaluation script is modified to include these changes, while the general structure of the inference loop is preserved.

As a first test, the images are infused with gradually increased gaussian noise before passing them to the policies, which are evaluated inside each of the four environments.



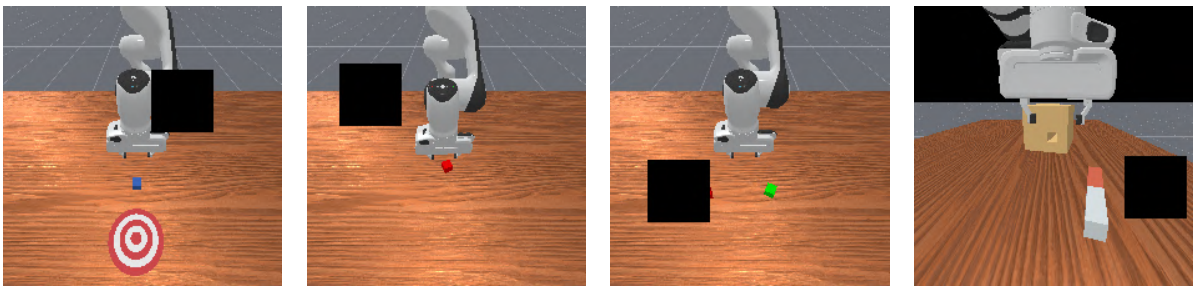
(a) Gaussian noise with $STD = 0.025$ (b) Gaussian noise with $STD = 0.05$ (c) Gaussian noise with $STD = 0.075$ (d) Gaussian noise with $STD = 0.1$

Figure 4.2: Camera images infused with noise.

The performance of Diffusion Policy, π_0 and SmolVLA gets increasingly worse the higher the standard deviation of the gaussian noise, with success rates that drops almost to zero

with STD higher then 0.1.

The second test introduced a visual occlusion, represented by a black patch of 50×50 pixels that is generated at a random position for each trial like in Figure 4.3.



(a) Push Cube (b) Pick Cube (c) Stack Cube (d) Peg Insertion Side

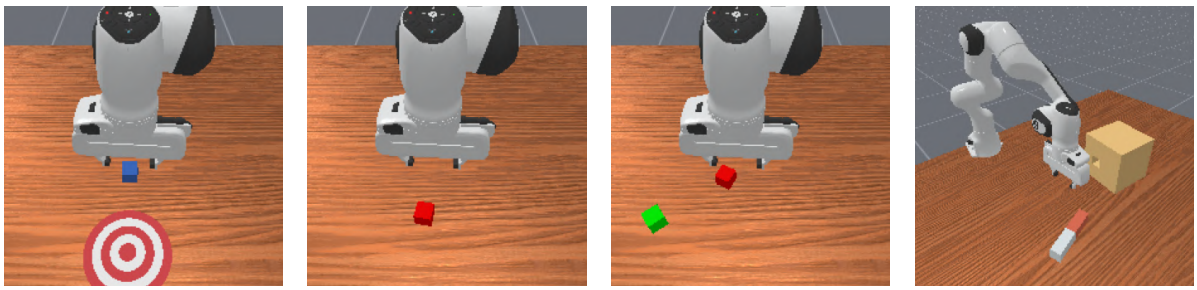
Figure 4.3: Occluded camera images.

In the cases of SmolVLA and Diffusion Policy, even when the occlusion was not covering the scene objects, they could not execute the task anymore.
However having the occlusion closer to the center of the view led to worse performance compared to when it was placed in the borders.

In the case of π_0, the policy did not seem to be impacted at all and was even able to execute the tasks when objects were partially covered. Including even one successful trial of "Peg Insertion".

Interestingly, when the objects are completely covered by the occlusion, all the policies still try to grasp somewhere within the occluded area, seemingly trying to guess the object's position.

The last test consists of modifying just the camera configuration so that the view of the scene is different. Either the field of view or the position and angulation of the camera are changed at a time, obtaining the images in Figure 4.4.



(a) Push Cube (b) Pick Cube (c) Stack Cube (d) Peg Insertion Side

Figure 4.4: Modified camera views.

This last test proved to be the most detrimental to performance as the robot arm clearly could not understand distances anymore, constantly underestimating them. Only "Push Cube" was occasionally solved successfully, while all the other task's success rates dropped to zero within all models.

5 | Conclusions, Limitations and Future Developments

5.1. Conclusions

The primary objective of this thesis was to conduct a rigorous comparative evaluation of four State-Of-The-Art generative AI architectures: Diffusion Policy, SmolVLA, π_0 and RDT-1B, to assess their capabilities and efficiency within constrained computing environments. By standardizing the training and evaluation protocols within the ManiSkill simulation framework, this work highlighted the distinct operational trade-offs of each model across four fundamental manipulation tasks: Push Cube, Pick Cube, Stack Cube, and Peg Insertion.

The empirical results reveal that there is no single universal solution, but rather models with specific strengths:

- **Diffusion Policy**: demonstrated the most natural and smooth robotic trajectories, excelling in multi-modal action precision and fast re-planning. Requires the least resources, both in terms of training time and hardware requirements, at the cost of being single-task, lacking semantic reasoning and broad generalization.
- **SmolVLA**: showed great ability in instruction and contextual understanding, reaching SOTA success rates, while leveraging a lighter model size. Sacrifices higher action accuracy and adaptability by prioritizing efficiency.
- $\boldsymbol{\pi_0}$: proved to be superior to SmolVLA in its generalization and accuracy capabilities with the downsides of a larger model, which consists in higher training and inference times.
- **RDT-1B**: resulted to be too resource-intensive for the constrained experimental setup, leading to failed task executions and extremely high latencies.

5.2. Limitations

Although generative AI models proved to be extraordinary tools, with unprecedented adaptability in robotics, they are currently limited by a multitude of factors.

- **Problem Solving**: Even if able to generalize well, these models do not really possess the ability to solve tasks that are too far from their training distribution or the ability to learn from their failures, evidenced by their tendency to get stuck in repetitive failure loops.
- **Resources Constraints**: The employment of this technology is still hampered by the need for expensive hardware, long training times and, above all, huge amounts of data, especially with an increasingly higher number of parameters per model.
- **Comparison Limitation**: It is objectively hard to fairly compare all the existing different architectures being developed continuously; and that is because of the huge amount of models already existing, the large quantities of independent variables and parameters that would need to be tested and all the different possible use-cases.

5.3. Future Developments

Concerning the thesis work, future developments should focus on the testing of more resource demanding models, like RDT-1B, in an appropriate environment, and then proceed to the employment of these models in the real world, bridging the simulation to reality gap.

From a more general point of view, research should now focus on reducing the resource constraints explained above, both for the applicability and sustainability of this technology, and then find a way so that these models do not only learn from successful demonstrations but also from their own failures.

5.4. Available Material

In the spirit of open science and to ensure the full reproducibility of the results achieved, all materials utilized in this thesis have been made openly available online.
Following, is provided a comprehensive list of references and links to get access to all the necessary resources.

In the GitHub repository, **"marco-porro/thesis"**, can be found all the scripts used to implement, train and evaluate the models, the scripts for dataset creation and conversion, the script for language embeddings pre-computing, the relative documentation for usability, and the links to all the Colab notebooks comprehensive of outputs and videos.
Link: `https://github.com/marco-porro/thesis`

In the Hugging Face user page, **"mimirmimir"** (Marco Porro), are uploaded all the datasets and obtained model checkpoints.
Link: `https://huggingface.co/mimirmimir`

Bibliography

[1] M. Ahn, A. Brohan, N. Brown, Y. Chebotar, O. Cortes, B. David, C. Finn, C. Fu, K. Gopalakrishnan, K. Hausman, A. Herzog, D. Ho, J. Hsu, J. Ibarz, B. Ichter, A. Irpan, E. Jang, R. J. Ruano, K. Jeffrey, S. Jesmonth, N. J. Joshi, R. Julian, D. Kalashnikov, Y. Kuang, K.-H. Lee, S. Levine, Y. Lu, L. Luu, C. Parada, P. Pastor, J. Quiambao, K. Rao, J. Rettinghouse, D. Reyes, P. Sermanet, N. Sievers, C. Tan, A. Toshev, V. Vanhoucke, F. Xia, T. Xiao, P. Xu, S. Xu, M. Yan, and A. Zeng. Do as i can, not as i say: Grounding language in robotic affordances. 2022. doi: 10.48550/arXiv.2204.01691.

[2] L. Beyer, A. Steiner, A. S. Pinto, A. Kolesnikov, X. Wang, D. Salz, M. Neumann, I. Alabdulmohsin, M. Tschannen, E. Bugliarello, T. Unterthiner, D. Keysers, S. Koppula, F. Liu, A. Grycner, A. Gritsenko, N. Houlsby, M. Kumar, K. Rong, J. Eisenschlos, R. Kabra, M. Bauer, M. Bošnjak, X. Chen, M. Minderer, P. Voigtlaender, I. Bica, I. Balazevic, J. Puigcerver, P. Papalampidi, O. Henaff, X. Xiong, R. Soricut, J. Harmsen, and X. Zhai. Paligemma: A versatile 3b vlm for transfer. 2024. doi: 10.48550/arXiv.2407.07726.

[3] K. Black, N. Brown, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, L. Groom, K. Hausman, B. Ichter, S. Jakubczak, T. Jones, L. Ke, S. Levine, A. Li-Bell, M. Mothukuri, S. Nair, K. Pertsch, L. X. Shi, J. Tanner, Q. Vuong, A. Walling, H. Wang, and U. Zhilinsky. A vision-language-action flow model for general robot control. 2024. doi: 10.48550/arXiv.2410.24164.

[4] R. Bommasani, D. A. Hudson, E. Adeli, R. Altman, S. Arora, S. von Arx, M. S. Bernstein, J. Bohg, A. Bosselut, E. Brunskill, E. Brynjolfsson, S. Buch, D. Card, R. Castellon, N. Chatterji, A. Chen, K. Creel, J. Q. Davis, D. Demszky, C. Donahue, M. Doumbouya, E. Durmus, S. Ermon, J. Etchemendy, K. Ethayarajh, L. Fei-Fei, C. Finn, T. Gale, L. Gillespie, K. Goel, N. Goodman, S. Grossman, N. Guha, T. Hashimoto, P. Henderson, J. Hewitt, D. E. Ho, J. Hong, K. Hsu, J. Huang, T. Icard, S. Jain, D. Jurafsky, P. Kalluri, S. Karamcheti, G. Keeling, F. Khani, O. Khattab, P. W. Koh, M. Krass, R. Krishna, R. Kuditipudi, A. Kumar, F. Ladhak, M. Lee, T. Lee,

J. Leskovec, I. Levent, X. L. Li, X. Li, T. Ma, A. Malik, C. D. Manning, S. Mirchandani, E. Mitchell, Z. Munyikwa, S. Nair, A. Narayan, D. Narayanan, B. Newman, A. Nie, J. C. Niebles, H. Nilforoshan, J. Nyarko, G. Ogut, L. Orr, I. Papadimitriou, J. S. Park, C. Piech, E. Portelance, C. Potts, A. Raghunathan, R. Reich, H. Ren, F. Rong, Y. Roohani, C. Ruiz, J. Ryan, C. Re, D. Sadigh, S. Sagawa, K. Santhanam, A. Shih, K. Srinivasan, A. Tamkin, R. Taori, A. W. Thomas, F. Tramer, R. E. Wang, W. Wang, B. Wu, J. Wu, Y. Wu, S. M. Xie, M. Yasunaga, J. You, M. Zaharia, M. Zhang, T. Zhang, X. Zhang, Y. Zhang, L. Zheng, K. Zhou, and P. Liang. On the opportunities and risks of foundation models. 2022. doi: 10.48550/arXiv.2108.07258.

[5] A. Brohan, N. Brown, J. Carbajal, Y. Chebotar, X. Chen, K. Choromanski, T. Ding, D. Driess, A. Dubey, C. Finn, P. Florence, C. Fu, M. Gonzalez Arenas, K. Gopalakrishnan, K. Han, K. Hausman, A. Herzog, J. Hsu, B. Ichter, A. Irpan, N. Joshi, R. Julian, D. Kalashnikov, Y. Kuang, I. Leal, L. Lee, T.-W. E. Lee, S. Levine, Y. Lu, H. Michalewski, I. Mordatch, K. Pertsch, K. Rao, K. Reymann, M. Ryoo, G. Salazar, P. Sanketi, P. Sermanet, J. Singh, A. Singh, R. Soricut, H. Tran, V. Vanhoucke, Q. Vuong, A. Wahid, S. Welker, P. Wohlhart, J. Wu, F. Xia, T. Xiao, P. Xu, S. Xu, T. Yu, and B. Zitkovich. Rt-2: Vision-language-action models transfer web knowledge to robotic control. 2023. doi: 10.48550/arXiv.2307.15818.

[6] A. Brohan, N. Brown, J. Carbajal, Y. Chebotar, J. Dabis, C. Finn, K. Gopalakrishnan, K. Hausman, A. Herzog, J. Hsu, J. Ibarz, B. Ichter, A. Irpan, T. Jackson, S. Jesmonth, N. J. Joshi, R. Julian, D. Kalashnikov, Y. Kuang, I. Leal, K.-H. Lee, S. Levine, Y. Lu, U. Malla, D. Manjunath, I. Mordatch, O. Nachum, C. Parada, J. Peralta, E. Perez, K. Pertsch, J. Quiambao, K. Rao, M. Ryoo, G. Salazar, P. Sanketi, K. Sayed, J. Singh, S. Sontakke, A. Stone, C. Tan, H. Tran, V. Vanhoucke, S. Vega, Q. Vuong, F. Xia, T. Xiao, P. Xu, S. Xu, T. Yu, and B. Zitkovich. Rt-1: Robotics transformer for real-world control at scale. 2023. doi: 10.48550/arXiv.2212.06817.

[7] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D. M. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen, E. Sigler, M. Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I. Sutskever, and D. Amodei. Language models are few-shot learners. 2020. doi: 10.48550/arXiv.2005.14165.

[8] J. Cen, C. Yu, H. Yuan, Y. Jiang, S. Huang, J. Guo, X. Li, Y. Song, H. Luo, F. Wang, D. Zhao, and H. Chen. Worldvla: Towards autoregressive action world model. 2025. doi: 10.48550/arXiv.2506.21539.

[9] C. Chi, Z. Xu, S. Feng, E. Cousineau, Y. Du, B. Burchfiel, R. Tedrake, and S. Song. Diffusion policy: Visuomotor policy learning via action diffusion. 2023. doi: 10.48550/arXiv.2303.04137.

[10] A. Chowdhery, S. Narang, J. Devlin, M. Bosma, G. Mishra, A. Roberts, P. Barham, H. W. Chung, C. Sutton, S. Gehrmann, P. Schuh, K. Shi, S. Tsvyashchenko, J. Maynez, A. Rao, P. Barnes, Y. Tay, N. Shazeer, V. Prabhakaran, E. Reif, N. Du, B. Hutchinson, R. Pope, J. Bradbury, J. Austin, M. Isard, G. Gur-Ari, P. Yin, T. Duke, A. Levskaya, S. Ghemawat, S. Dev, H. Michalewski, X. Garcia, V. Misra, K. Robinson, L. Fedus, D. Zhou, D. Ippolito, D. Luan, H. Lim, B. Zoph, A. Spiridonov, R. Sepassi, D. Dohan, S. Agrawal, M. Omernick, A. M. Dai, T. S. Pillai, M. Pellat, A. Lewkowycz, E. Moreira, R. Child, O. Polozov, K. Lee, Z. Zhou, X. Wang, B. Saeta, M. Diaz, O. Firat, M. Catasta, J. Wei, K. Meier-Hellstern, D. Eck, J. Dean, S. Petrov, and N. Fiedel. Palm: Scaling language modeling with pathways. 2022. doi: 10.48550/arXiv.2204.02311.

[11] O. X.-E. Collaboration, A. O'Neill, A. Rehman, A. Gupta, A. Maddukuri, A. Gupta, A. Padalkar, A. Lee, A. Pooley, A. Gupta, A. Mandlekar, A. Jain, A. Tung, A. Bewley, A. Herzog, A. Irpan, A. Khazatsky, A. Rai, A. Gupta, A. Wang, A. Kolobov, A. Singh, A. Garg, A. Kembhavi, A. Xie, A. Brohan, A. Raffin, A. Sharma, A. Yavary, A. Jain, A. Balakrishna, A. Wahid, B. Burgess-Limerick, B. Kim, B. Schölkopf, B. Wulfe, B. Ichter, C. Lu, C. Xu, C. Le, C. Finn, C. Wang, C. Xu, C. Chi, C. Huang, C. Chan, C. Agia, C. Pan, C. Fu, C. Devin, D. Xu, D. Morton, D. Driess, D. Chen, D. Pathak, D. Shah, D. Büchler, D. Jayaraman, D. Kalashnikov, D. Sadigh, E. Johns, E. Foster, F. Liu, F. Ceola, F. Xia, F. Zhao, F. V. Frujeri, F. Stulp, G. Zhou, G. S. Sukhatme, G. Salhotra, G. Yan, G. Feng, G. Schiavi, G. Berseth, G. Kahn, G. Yang, G. Wang, H. Su, H.-S. Fang, H. Shi, H. Bao, H. B. Amor, H. I. Christensen, H. Furuta, H. Bharadhwaj, H. Walke, H. Fang, H. Ha, I. Mordatch, I. Radosavovic, I. Leal, J. Liang, J. Abou-Chakra, J. Kim, J. Drake, J. Peters, J. Schneider, J. Hsu, J. Vakil, J. Bohg, J. Bingham, J. Wu, J. Gao, J. Hu, J. Wu, J. Wu, J. Sun, J. Luo, J. Gu, J. Tan, J. Oh, J. Wu, J. Lu, J. Yang, J. Malik, J. Silvério, J. Hejna, J. Booher, J. Tompson, J. Yang, J. Salvador, J. J. Lim, J. Han, K. Wang, K. Rao, K. Pertsch, K. Hausman, K. Go, K. Gopalakrishnan, K. Goldberg, K. Byrne, K. Oslund, K. Kawaharazuka, K. Black, K. Lin, K. Zhang, K. Ehsani, K. Lekkala, K. Ellis, K. Rana, K. Srinivasan, K. Fang, K. P. Singh, K.-H. Zeng, K. Hatch, K. Hsu, L. Itti, L. Y. Chen, L. Pinto, L. Fei-Fei, L. Tan, L. Fan, L. Ott, L. Lee, L. Weihs, M. Chen, M. Lepert, M. Memmel, M. Tomizuka, M. Itkina, M. G. Castro, M. Spero, M. Du, M. Ahn, M. C. Yip, M. Zhang, M. Ding, M. Heo,

M. K. Srirama, M. Sharma, M. J. Kim, M. Z. Irshad, N. Kanazawa, N. Hansen, N. Heess, N. J. Joshi, N. Suenderhauf, N. Liu, N. Di Palo, N. M. M. Shafiullah, O. Mees, O. Kroemer, O. Bastani, P. R. Sanketi, P. Miller, P. Yin, P. Wohlhart, P. Xu, P. D. Fagan, P. Mitrano, P. Sermanet, P. Abbeel, P. Sundaresan, Q. Chen, Q. Vuong, R. Rafailov, R. Tian, R. Doshi, R. Martín-Martín, R. Baijal, R. Scalise, R. Hendrix, R. Lin, R. Qian, R. Zhang, R. Mendonca, R. Shah, R. Hoque, R. Julian, S. Bustamante, S. Kirmani, S. Levine, S. Lin, S. Moore, S. Bahl, S. Dass, S. Sonawani, S. Tulsiani, S. Song, S. Xu, S. Haldar, S. Karamcheti, S. Adebola, S. Guist, S. Nasiriany, S. Schaal, S. Welker, S. Tian, S. Ramamoorthy, S. Dasari, S. Belkhale, S. Park, S. Nair, S. Mirchandani, T. Osa, T. Gupta, T. Harada, T. Matsushima, T. Xiao, T. Kollar, T. Yu, T. Ding, T. Davchev, T. Z. Zhao, T. Armstrong, T. Darrell, T. Chung, V. Jain, V. Kumar, V. Vanhoucke, V. Guizilini, W. Zhan, W. Zhou, W. Burgard, X. Chen, X. Chen, X. Wang, X. Zhu, X. Geng, X. Liu, L. Xu, X. Li, Y. Pang, Y. Lu, Y. J. Ma, Y. Kim, Y. Chebotar, Y. Zhou, Y. Zhu, Y. Wu, Y. Xu, Y. Wang, Y. Bisk, Y. Dou, Y. Cho, Y. Lee, Y. Cui, Y. Cao, Y.-H. Wu, Y. Tang, Y. Zhu, Y. Zhang, Y. Jiang, Y. Li, Y. Li, Y. Iwasawa, Y. Matsuo, Z. Ma, Z. Xu, Z. J. Cui, Z. Zhang, Z. Fu, and Z. Lin. Open x-embodiment: Robotic learning datasets and rt-x models. 2025. doi: 10.48550/arXiv.2310.08864.

[12] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. 2019. doi: 10.48550/arXiv.1810.04805.

[13] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale. 2021. doi: 10.48550/arXiv.2010.11929.

[14] P. Esser, S. Kulal, A. Blattmann, R. Entezari, J. Müller, H. Saini, Y. Levi, D. Lorenz, A. Sauer, F. Boesel, D. Podell, T. Dockhorn, Z. English, K. Lacey, A. Goodwin, Y. Marek, and R. Rombach. Scaling rectified flow transformers for high-resolution image synthesis. 2024. doi: 10.48550/arXiv.2403.03206.

[15] J. Gui, T. Chen, J. Zhang, Q. Cao, Z. Sun, H. Luo, and D. Tao. A survey on self-supervised learning: Algorithms, applications, and future trends. 2024. doi: 10.48550/arXiv.2301.05712.

[16] J. Ho, A. Jain, and P. Abbeel. Denoising diffusion probabilistic models. 2020. doi: 10.48550/arXiv.2006.11239.

[17] S. Hochreiter and J. Schmidhuber. Long short-term memory. 1997. doi: 10.1162/neco.1997.9.8.1735.

[18] P. Holderrieth and E. Erives. An introduction to flow matching and diffusion models. 2025. doi: 10.48550/arXiv.2506.02070.

[19] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen. Lora: Low-rank adaptation of large language models. 2021. doi: 10.48550/arXiv.2106.09685.

[20] P. Intelligence, K. Black, N. Brown, J. Darpinian, K. Dhabalia, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, M. Y. Galliker, D. Ghosh, L. Groom, K. Hausman, B. Ichter, S. Jakubczak, T. Jones, L. Ke, D. LeBlanc, S. Levine, A. Li-Bell, M. Mothukuri, S. Nair, K. Pertsch, A. Z. Ren, L. X. Shi, L. Smith, J. T. Springenberg, K. Stachowicz, J. Tanner, Q. Vuong, H. Walke, A. Walling, H. Wang, L. Yu, and U. Zhilinsky. $\pi_{0.5}$: A vision-language-action model with open-world generalization. 2025. doi: 10.48550/arXiv.2504.16054.

[21] S. James, Z. Ma, D. R. Arrojo, and A. J. Davison. Rlbench: The robot learning benchmark & learning environment. 2019. doi: 10.48550/arXiv.1909.12271.

[22] R. Jiang, G.-C. Zheng, T. Li, T.-R. Yang, J.-D. Wang, and X. Li. A survey of multimodal controllable diffusion models. 2024. doi: 10.1007/s11390-024-3814-0.

[23] Y. Jiang, A. Gupta, Z. Zhang, G. Wang, Y. Dou, Y. Chen, L. Fei-Fei, A. Anandkumar, Y. Zhu, and L. Fan. Vima: General robot manipulation with multimodal prompts. 2023. doi: 10.48550/arXiv.2210.03094.

[24] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Rafailov, E. Foster, G. Lam, P. Sanketi, Q. Vuong, T. Kollar, B. Burchfiel, R. Tedrake, D. Sadigh, S. Levine, P. Liang, and C. Finn. Openvla: An open-source vision-language-action model. 2024. doi: 10.48550/arXiv.2406.09246.

[25] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, and M. Le. Flow matching for generative modeling. 2023. doi: 10.48550/arXiv.2210.02747.

[26] S. Liu, L. Wu, B. Li, H. Tan, H. Chen, Z. Wang, K. Xu, H. Su, and J. Zhu. Rdt-1b: A diffusion foundation model for bimanual manipulation. 2024. doi: 10.48550/arXiv.2410.07864.

[27] X. Liu, C. Gong, and Q. Liu. Flow straight and fast: Learning to generate and transfer data with rectified flow. 2022. doi: 10.48550/arXiv.2209.03003.

[28] V. Makoviychuk, L. Wawrzyniak, Y. Guo, M. Lu, K. Storey, M. Macklin, D. Hoeller, N. Rudin, A. Allshire, A. Handa, and G. State. Isaac gym: High performance gpu-based physics simulation for robot learning. 2021. doi: 10.48550/arXiv.2108.10470.

[29] T. Mu, Z. Ling, F. Xiang, D. Yang, X. Li, S. Tao, Z. Huang, Z. Jia, and H. Su. Maniskill: Generalizable manipulation skill benchmark with large-scale demonstrations. 2021. doi: 10.48550/arXiv.2107.14483.

[30] M. Oquab, T. Darcet, T. Moutakanni, H. Vo, M. Szafraniec, V. Khalidov, P. Fernandez, D. Haziza, F. Massa, A. El-Nouby, M. Assran, N. Ballas, W. Galuba, R. Howes, P.-Y. Huang, S.-W. Li, I. Misra, M. Rabbat, V. Sharma, G. Synnaeve, H. Xu, H. Jegou, J. Mairal, P. Labatut, A. Joulin, and P. Bojanowski. Dinov2: Learning robust visual features without supervision. 2024. doi: 10.48550/arXiv.2304.07193.

[31] K. Pertsch, K. Stachowicz, B. Ichter, D. Driess, S. Nair, Q. Vuong, O. Mees, C. Finn, and S. Levine. Fast: Efficient action tokenization for vision-language-action models. 2025. doi: 10.48550/arXiv.2501.09747.

[32] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, G. Krueger, and I. Sutskever. Learning transferable visual models from natural language supervision. 2021. doi: 10.48550/arXiv.2103.00020.

[33] S. Reed, K. Zolna, E. Parisotto, S. Gomez Colmenarejo, A. Novikov, G. Barth-Maron, M. Gimenez, Y. Sulsky, J. Kay, J. T. Springenberg, T. Eccles, J. Bruce, A. Razavi, A. Edwards, N. Heess, Y. Chen, R. Hadsell, O. Vinyals, M. Bordbar, and N. de Freitas. A generalist agent. 2022. doi: 10.48550/arXiv.2205.06175.

[34] O. Ronneberger, P. Fischer, and T. Brox. U-net: Convolutional networks for biomedical image segmentation. 2015. doi: 10.48550/arXiv.1505.04597.

[35] D. E. Rumelhart, G. E. Hinton, and R. J. Williams. Learning internal representations by error propagation. 1986.

[36] O. Russakovsky, J. Deng, H. Su, J. Krause, S. Satheesh, S. Ma, Z. Huang, A. Karpathy, A. Khosla, M. Bernstein, A. C. Berg, and L. Fei-Fei. Imagenet large scale visual recognition challenge. 2015. doi: 10.48550/arXiv.1409.0575.

[37] M. Shukor, D. Aubakirova, F. Capuano, P. Kooijmans, S. Palma, A. Zouitine, M. Aractingi, C. Pascal, M. Russi, A. Marafioti, S. Alibert, M. Cord, T. Wolf, and R. Cadene. Smolvla: A vision-language-action model for affordable and efficient robotics. 2025. doi: 10.48550/arXiv.2506.01844.

[38] M. Tan and Q. V. Le. Efficientnet: Rethinking model scaling for convolutional neural networks. 2020. doi: 10.48550/arXiv.1905.11946.

[39] S. Tao, F. Xiang, A. Shukla, Y. Qin, X. Hinrichsen, X. Yuan, C. Bao, X. Lin, Y. Liu, T.-k. Chan, Y. Gao, X. Li, T. Mu, N. Xiao, A. Gurha, V. N. Rajesh, Y. W. Choi, Y.-R. Chen, Z. Huang, R. Calandra, R. Chen, S. Luo, and H. Su. Maniskill3: Gpu parallelized robotics simulation and rendering of generalizable embodied ai. 2025. doi: 10.48550/arXiv.2410.00425.

[40] G. Team, T. Mesnard, C. Hardin, R. Dadashi, S. Bhupatiraju, S. Pathak, L. Sifre, M. Rivière, M. S. Kale, J. Love, P. Tafti, L. Hussenot, P. G. Sessa, A. Chowdhery, A. Roberts, A. Barua, A. Botev, A. Castro-Ros, A. Slone, A. Héliou, A. Tacchetti, A. Bulanova, A. Paterson, B. Tsai, B. Shahriari, C. L. Lan, C. A. Choquette-Choo, C. Crepy, D. Cer, D. Ippolito, D. Reid, E. Buchatskaya, E. Ni, E. Noland, G. Yan, G. Tucker, G.-C. Muraru, G. Rozhdestvenskiy, H. Michalewski, I. Tenney, I. Grishchenko, J. Austin, J. Keeling, J. Labanowski, J.-B. Lespiau, J. Stanway, J. Brennan, J. Chen, J. Ferret, J. Chiu, J. Mao-Jones, K. Lee, K. Yu, K. Millican, L. L. Sjoesund, L. Lee, L. Dixon, M. Reid, M. Mikuła, M. Wirth, M. Sharman, N. Chinaev, N. Thain, O. Bachem, O. Chang, O. Wahltinez, P. Bailey, P. Michel, P. Yotov, R. Chaabouni, R. Comanescu, R. Jana, R. Anil, R. McIlroy, R. Liu, R. Mullins, S. L. Smith, S. Borgeaud, S. Girgin, S. Douglas, S. Pandya, S. Shakeri, S. De, T. Klimenko, T. Hennigan, V. Feinberg, W. Stokowiec, Y.-h. Chen, Z. Ahmed, Z. Gong, T. Warkentin, L. Peran, M. Giang, C. Farabet, O. Vinyals, J. Dean, K. Kavukcuoglu, D. Hassabis, Z. Ghahramani, D. Eck, J. Barral, F. Pereira, E. Collins, A. Joulin, N. Fiedel, E. Senter, A. Andreev, and K. Kenealy. Gemma: Open models based on gemini research and technology. 2024. doi: 10.48550/arXiv.2403.08295.

[41] O. M. Team, D. Ghosh, H. Walke, K. Pertsch, K. Black, O. Mees, S. Dasari, J. Hejna, T. Kreiman, C. Xu, J. Luo, Y. L. Tan, L. Y. Chen, P. Sanketi, Q. Vuong, T. Xiao, D. Sadigh, C. Finn, and S. Levine. Octo: An open-source generalist robot policy. 2024. doi: 10.48550/arXiv.2405.12213.

[42] F. Torabi, G. Warnell, and P. Stone. Behavioral cloning from observation. 2018. doi: 10.48550/arXiv.1805.01954.

[43] H. Touvron, L. Martin, K. Stone, P. Albert, A. Almahairi, Y. Babaei, N. Bashlykov, S. Batra, P. Bhargava, S. Bhosale, D. Bikel, L. Blecher, C. Canton Ferrer, M. Chen, G. Cucurull, D. Esiobu, J. Fernandes, J. Fu, W. Fu, B. Fuller, C. Gao, V. Goswami, N. Goyal, A. Hartshorn, S. Hosseini, R. Hou, H. Inan, M. Kardas, V. Kerkez, M. Khabsa, I. Kloumann, A. Korenev, P. S. Koura, M.-A. Lachaux,

T. Lavril, J. Lee, D. Liskovich, Y. Lu, Y. Mao, X. Martinet, T. Mihaylov, P. Mishra, I. Molybog, Y. Nie, A. Poulton, J. Reizenstein, R. Rungta, K. Saladi, A. Schelten, R. Silva, E. M. Smith, R. Subramanian, X. E. Tan, B. Tang, R. Taylor, A. Williams, J. X. Kuan, P. Xu, Z. Yan, I. Zarov, Y. Zhang, A. Fan, M. Kambadur, S. Narang, A. Rodriguez, R. Stojnic, S. Edunov, and T. Scialom. Llama 2: Open foundation and fine-tuned chat models. 2023. doi: 10.48550/arXiv.2307.09288.

[44] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin. Attention is all you need. 2017. doi: 10.48550/arXiv.1706.03762.

[45] L. Wang, D. Liu, X. Liu, and X. He. Scaling down text encoders of text-to-image diffusion models. 2025. doi: 10.48550/arXiv.2503.19897.

[46] J. Wen, Y. Zhu, J. Li, M. Zhu, K. Wu, Z. Xu, N. Liu, R. Cheng, C. Shen, Y. Peng, F. Feng, and J. Tang. Tinyvla: Towards fast, data-efficient vision-language-action models for robotic manipulation. 2025. doi: 10.48550/arXiv.2409.12514.

[47] Y. Weng, X. Zhang, Y. Mu, Y. Zhu, Y. Li, and Q. Liu. Temporal action selection for action chunking. 2025. doi: 10.48550/arXiv.2511.04421.

[48] K. Zakka, B. Tabanpour, Q. Liao, M. Haiderbhai, S. Holt, J. Y. Luo, A. Allshire, E. Frey, K. Sreenath, L. A. Kahrs, C. Sferrazza, Y. Tassa, and P. Abbeel. Mujoco playground. 2025. doi: 10.48550/arXiv.2502.08844.

[49] Y. Ze, G. Zhang, K. Zhang, C. Hu, M. Wang, and H. Xu. 3d diffusion policy: Generalizable visuomotor policy learning via simple 3d representations. 2024. doi: 10.48550/arXiv.2403.03954.

[50] X. Zhai, B. Mustafa, A. Kolesnikov, and L. Beyer. Sigmoid loss for language image pre-training. 2023. doi: 10.48550/arXiv.2303.15343.

A | Benchmarks Leaderboard

Table A.1: Average success rates across different official benchmarks.

Model Name	LIBERO	Meta-World	CALVIN	LIBERO+	ROBOTWIN 2.0
SimpleVLA-RL	99.1				
Being-H0.5 (Specialist)	98.9				
UnifoLM-VLA-0	98.7				
VLA-Adapter-pro	98.5		4.50		
pi-RL	98.3	85.8	4.71		
EO-1	98.2				
X-VLA	98.1		4.43		
RLinf-VLA	98.1				
VOTE	98.0				
RIPT-VLA	97.5			68.4	
VLA-Cache	97.4				
OpenVLA-OFT	97.1			69.6	
CronusVLA	97.0				
DeepThinkVLA	97.0			79.0	
Flower VLA	96.9		4.53		
MemoryVLA	96.5				
Discrete Diffusion VLA	96.3				
InternVLA-M1	95.9				
InstructVLA	95.8				
UnifiedVLA	95.5		4.41		
UniVLA	95.2			43.9	
NORA-1.5 (DPO)	95.0				
Evo-1	94.8	80.6			
BitVLA	94.8				
VLA-0	94.7				
pi0.5-KI	94.3				
pi0	94.2	47.9		53.6	31.4
GR00T-N1	93.9				
DreamVLA	92.6		4.44		
SmolVLA	88.8	57.3			
NORA	87.9			39.0	
MolmoAct	86.6				
pi0-FAST	85.5			61.6	
Dita	82.4		3.61		
WorldVLA	81.8			25.0	
VLA-RL	81.0				
GRAPE	80.2				
SpatialVLA	78.1				
OpenVLA	76.5			15.6	
Octo	75.1				
TraceVLA	74.8				
UniAct	74.0				
TTF-VLA	74.0				
DP3 [49]					30.1
RDT-1B					24.1
TinyVLA		31.6			
RT-1			0.90		
ACT					15.7
Diffusion Policy	72.4	10.5	0.56		14.3

B | Hyperparameters

Table B.1: Hyperparameters and configuration details for Diffusion Policy training.

Parameter	**Value**
Architecture & Backbone	
Base Model	Diffusion Policy (CNN-based)
Vision Backbone	ResNet-18 (Trained from scratch)
Trainable Parameters	Full Model
Observation & Action Space	
Input Image Resolution	224×224 / 84×84
Image Resolution Crop	200×200 / 76×76
Observation History	2 frames
Action Horizon (Chunking)	16 steps
Action Steps Executed	8 steps
Optimization	
Optimizer	Adam ($\beta_1 = 0.95, \beta_2 = 0.999, \epsilon = 10^{-8}$)
Peak Learning Rate	1×10^{-4}
LR Schedule	Cosine 500 warmup steps
Batch Size	64
Total Training Steps	170,000 / 335,000 (both 1,000 epochs)
Weight Decay	1×10^{-6}
Gradient Clipping Norm	10.0
Precision	Float32 (AMP Disabled)
Diffusion Parameters	
Noise Scheduler	DDPM
Train Timesteps	100
Beta Schedule	`squaredcos_cap_v2` ($\beta \in [10^{-4}, 0.02]$)
Step Embedding Dimension	128

Table B.2: Hyperparameters and configuration details for SmolVLA training.

Parameter	Value
Architecture & Backbone	
Base Model	SmolVLM2-500M-Video-Instruct
Vision Encoder	Frozen
Trainable Parameters	Action Expert & State Projection
Observation & Action Space	
Input Image Resolution	224×224
Tokenization Resolution	512×512 (padded)
Action Horizon (Chunking)	50 steps
Observation History	1 frame
Optimization	
Optimizer	AdamW ($\beta_1 = 0.9, \beta_2 = 0.95$)
Peak Learning Rate	1×10^{-4}
LR Schedule	Cosine Decay w/ 100 warmup steps
Batch Size	64
Total Training Steps	40,000
Weight Decay	1×10^{-10}
Precision	Automatic Mixed Precision (AMP)
Flow Matching & Diffusion	
Inference Steps	10

Table B.3: Hyperparameters and configuration details for π_0 training.

Parameter	Value
Architecture & Backbone	
Base Model	π_0 (PaliGemma-2B + Gemma-300M)
Vision Encoder	Frozen (40k) + Unfrozen (20k)
Trainable Parameters	Action Expert Only (40k) + All (20k)
Observation & Action Space	
Input Image Resolution	224×224
Action Horizon (Chunking)	16 steps
Observation History	1 frame
Optimization	
Optimizer	AdamW ($\beta_1 = 0.9, \beta_2 = 0.95, \epsilon = 10^{-8}$)
Peak Learning Rate	2.5×10^{-5}
LR Schedule	Cosine Decay w/ 1,000 warmup steps
Batch Size	32
Total Training Steps	40,000+20,000
Weight Decay	0.01
Precision	`bfloat16`
Flow Matching & Diffusion	
Inference Steps	10

Table B.4: Hyperparameters and configuration details for RDT-1B training.

Parameter	**Value**
Architecture & Backbone	
Base Model	RDT-1B
Vision Encoder	SigLIP-SO400M (`patch14-384`)
Precomputed Lang Embeddings	True
Observation & Action Space	
Action Horizon (Chunking)	64 steps
Conditioning Mask Probability	0.1
Optimization	
Optimizer	AdamW ($\beta_1 = 0.9, \beta_2 = 0.999, \epsilon = 10^{-8}$)
Peak Learning Rate	1×10^{-4}
LR Schedule	Cosine w/ 500 warmup steps
Batch Size	8 (w/ Gradient Accumulation 4)
Total Training Steps	5,000 (385 epochs)
Weight Decay	1×10^{-2}
Gradient Clipping Norm	1.0
Precision	`bfloat16` (DeepSpeed ZeRO-2)
Flow Matching & Diffusion	
Noise Scheduler	DDPM
Train Timesteps	1,000
Inference Steps	5
Beta Schedule	`squaredcos_cap_v2`

List of Figures

1 General-Purpose robotics workflow. 1

1.1 Transformer Architecture. 6
1.2 Graphical representation of the diffusion process [22]. 8
1.3 Graphical interpretation of denoising steps comparison. 10
1.4 π_0 architecture. 11
1.5 SmolVLA architecture. 12
1.6 RDT-1B architecture. 12

2.1 ManiSkill tasks render 18
2.2 ManiSkill tasks base camera view 19

3.1 Diffusion Policy training metrics. 24
3.2 Diffusion Policy training metrics with paper dataset configuration. 24
3.3 SmolVLA training metrics. 26
3.4 Pi0 training metrics. 27
3.5 RDT-1B training metrics. 29

4.1 Diffusion Policy image resolution comparison. 34
4.2 Camera images infused with noise. 39
4.3 Occluded camera images. 40
4.4 Modified camera views. 41

List of Tables

1.1 Average success rates (%) across different benchmarks and papers. 13

3.1 Action Accuracy Test of Diffusion Policy 25
3.2 Evaluation Success Rates of Diffusion Policy 25
3.3 Action Accuracy Test of SmolVLA 26
3.4 Evaluation Success Rates of SmolVLA 26
3.5 Action Accuracy Test of π_0 28
3.6 Evaluation Success Rates of π_0 28
3.7 Comparative evaluation success rates (%) across different models. 30
3.8 Training specifications and resource requirements. 30

4.1 Peg Insertion success rates with variable clearance. 37

A.1 Average success rates across different official benchmarks. 55

B.1 Hyperparameters and configuration details for Diffusion Policy training. . . 57
B.2 Hyperparameters and configuration details for SmolVLA training. 58
B.3 Hyperparameters and configuration details for π_0 training. 58
B.4 Hyperparameters and configuration details for RDT-1B training. 59